Riana Matengu
Student ID: 2231147
BSc honors in Network security and computer forensics
Chapter 1, 2 and 3 final

Risk-Based Vulnerability Assessment and Management System for Prioritizing Cybersecurity Threats

Chapters 1, 2 and 3

Riana Matengu | Student: Network Security and Computer Forensics

Table of Contents

Chapter 1: Project Proposal....4

1.1 Aim and Objectives.... 4
1.2 Expected Outcomes / Deliverables....4
1.3 Keywords....5
1.4 Introduction / Background / Overview....5
1.5 Problem Statement....5
1.6 Proposed Solution....6
1.7 Project Scope.... 6
1.8 Ethical Consideration....7
1.9 Project Type....7
1.10 Research Methodology.... 7
1.11 Resource Requirements.... 8
1.12 Project Plan....9

Chapter 2: First Review....10

2.1 Literature Review.... 10
2.2 Research Methodology.... 13
2.3 Resource Requirements.... 14
2.4 Functional Requirements / Non-Functional Requirements.... 14
2.5 Modeling Diagrams.... 15
2.6 Project Plan....17

Chapter 3: Second Review.... 18

3.1 UML Design Guidelines.... 18
3.2 Class Diagram.... 18
3.3 Object Diagrams....19
3.4 Sequence Diagram.... 20
3.5 Communication Diagram....21
3.6 State Charts.... 22
3.7 Process Design.... 23

Chapter 4: Final Project.... 25

4.1 Implementation of the design.... 25
4.2 Testing techniques.... 38
4.3 Project evaluation.... 59

References.... 61

Chapter 1: Project Proposal

1.1 Aim and Objectives

1.1.1 Aim

To design and develop a Risk-Based Vulnerability Assessment and Management System that enables organizations to identify, assess, prioritize, and manage cybersecurity vulnerabilities by combining technical vulnerability data with organizational risk factors.

1.1.2 Objectives

- To study existing vulnerability assessment and management approaches based on the Common Vulnerabilities and Exposures (CVE) database and the Common Vulnerability Scoring System (CVSS).
- To identify the key factors influencing vulnerability prioritization, including asset criticality, exploit availability, organizational exposure, and remediation status.
- To design a normalized relational SQL database capable of storing vulnerability, asset, remediation, and user information.
- To develop a system that calculates a custom organizational risk score, the Smart Priority Score, for each vulnerability using predefined weighting criteria.
- To implement a dashboard that assists security administrators in prioritizing vulnerabilities for remediation.
- To test and evaluate the effectiveness of the proposed system in improving vulnerability management and remediation prioritization.

1.2 Expected Outcomes / Deliverables

At the completion of the project, the deliverable submitted will be the working artefact itself: the complete Risk-Based Vulnerability Assessment and Management System described in Sections 1.1 and 1.6, combining the database, the NVD integration, the Smart Priority Score engine, and the dashboard into a single functioning system rather than as separate component deliverables.

Alongside the artefact, full supporting documentation will be submitted, covering the database design, system architecture, user manual, and technical report, together with test results demonstrating the system's effectiveness and reliability.

1.3 Keywords

Vulnerability Management, CVSS (Common Vulnerability Scoring System), Risk-Based Prioritization, Asset Management, Remediation.

1.4 Introduction / Background / Overview

Cybersecurity threats continue to increase in both frequency and sophistication, making vulnerability management a critical component of organizational security. Organizations often rely on publicly available vulnerability databases such as CVE and CVSS to identify security weaknesses within their information systems. Although these resources provide standardized vulnerability information and severity scores, they do not consider an organization's unique environment when determining remediation priorities.

Many organizations face thousands of vulnerabilities across different systems, making it difficult to determine which vulnerabilities should be addressed first. A vulnerability with a high CVSS score may affect a low-value asset, while a medium-severity vulnerability affecting a critical business system may pose a significantly greater risk.

This project proposes the development of a Risk-Based Vulnerability Assessment and Management System that enhances traditional vulnerability management by incorporating organizational context into vulnerability prioritization. Instead of relying solely on CVSS severity scores, the system will evaluate multiple risk factors, including asset importance, exploit availability, exposure level, patch availability, and remediation status, to calculate a customized organizational risk score.

The proposed solution will provide security administrators with actionable information that supports informed decision-making and efficient allocation of remediation resources.

1.5 Problem Statement

Traditional vulnerability management approaches prioritize vulnerabilities primarily using standardized severity scores such as CVSS. While these scores provide valuable technical information, they fail to account for organizational factors, including asset criticality, exposure to external threats, exploit availability, and business impact.

As a result, organizations often allocate resources inefficiently by remediating vulnerabilities based solely on severity rather than actual business risk. This may leave critical organizational assets exposed while lower-priority vulnerabilities receive unnecessary attention.

There is therefore a need for a vulnerability management system that combines technical vulnerability information with organizational risk factors to improve vulnerability prioritization and support more effective cybersecurity decision-making.

1.6 Proposed Solution

The proposed solution is the development of a Risk-Based Vulnerability Assessment and Management System that integrates vulnerability information with organizational asset data to generate a Smart Priority Score for each identified vulnerability.

The system will maintain information regarding vulnerabilities, affected assets, remediation activities, and user responsibilities within a centralized SQL database.

The Smart Priority Score will be calculated using several factors, including:

- CVSS severity score
- Asset criticality
- Exploit availability
- Public exploitation status
- Organizational exposure
- Number of affected systems
- Patch availability
- Remediation progress

Based on these factors, vulnerabilities will be classified into different risk levels, enabling administrators to prioritize remediation activities more effectively.

The system will also provide dashboards, reports, audit logs, and workflow management features that support vulnerability tracking from initial discovery through verification and final closure.

1.7 Project Scope

This project focuses on the design and development of a single-organization Risk-Based Vulnerability Assessment and Management System within the four-month module timeframe. In scope: relational database design for vulnerability, asset, remediation, and user data; a live data feed from the National Vulnerability Database (NVD) API for CVE and CVSS information; the Smart Priority Score calculation engine; a web-based dashboard with authentication and role-based access; remediation workflow tracking; and system testing using sample organizational scenarios.

Out of scope: integration with live production vulnerability scanners (e.g. Nessus, Qualys, OpenVAS), since no such scanner infrastructure is available within the project timeframe; multi-tenant support for more than one organization; and automated remediation or patch deployment, since the system is designed to prioritize and track remediation rather than perform it directly. The system is a proof-of-concept that could be extended to integrate with live scanners and automated patching pipelines in future work.

1.8 Ethical Consideration

This project does not involve human research participants, so no personal data will be collected from individuals in the course of the research itself. All vulnerability, asset, and organizational data used to test and demonstrate the system will be fabricated or drawn from publicly available vulnerability records (CVE/CVSS entries from the NVD), rather than from any real organization's actual infrastructure or security posture, to avoid exposing real security weaknesses.

Where sample user accounts are required to demonstrate authentication and role-based access control, only fabricated, non-identifying credentials will be used. Access to the National Vulnerability Database API will be conducted strictly in accordance with NVD's terms of use and published rate limits.

1.9 Project Type

This project is primarily a software development project supported by applied research.

The research component will investigate current vulnerability management practices and existing cybersecurity frameworks, while the development component will involve the design, implementation, testing, and evaluation of the proposed vulnerability management system.

1.10 Research Methodology

The project will adopt a Design Science Research methodology combined with an Iterative Software Development Life Cycle (SDLC) model, in which the database, backend logic, and dashboard are each designed, built, and tested in short cycles rather than all at once.

The research activities will include:

- Reviewing academic literature on vulnerability assessment and cybersecurity risk management.
- Studying existing standards such as CVE and CVSS, and complementary prioritization signals such as EPSS and CISA KEV.
- Gathering system requirements through document analysis of published vulnerability management frameworks and guidance.
- Designing the database using Entity Relationship Diagrams (ERDs).
- Developing the application using SQL and an appropriate programming language and framework.
- Testing system functionality through unit, integration, and user acceptance testing.
- Evaluating the effectiveness of the Smart Priority Score by comparing prioritized vulnerabilities under different organizational scenarios.

1.11 Resource Requirements

The successful completion of this project will require the following resources:

Hardware

- Personal computer or laptop (minimum 8GB RAM recommended, to comfortably run the database server, application server, and IDE concurrently during development).
- Stable internet connection (required for NVD API access and version control).

Software

The following software stack has been selected for this project, rather than left as an open set of alternatives, so that development can begin immediately with a consistent, compatible toolchain.

Category	Tool / Technology	Purpose
Backend framework	Python 3.11+ with Django 5	Application logic, ORM, built-in authentication, and rapid secure development of the dashboard and API.
Database	MySQL 8.0 (Community Edition)	Relational storage for vulnerability, asset, remediation, and user/audit data.

Category	Tool / Technology	Purpose
Frontend	Django Templates + Bootstrap 5 + Chart.js	Dashboard UI and visual charts of prioritized vulnerabilities and risk levels.
Vulnerability data source	National Vulnerability Database (NVD) REST API 2.0	Live import of real CVE records and CVSS base scores rather than manually entered data.
IDE	Visual Studio Code	Primary development environment for backend, frontend, and scripting.
Version control	Git and GitHub	Source control, change history, and backup of the codebase.
Diagramming	Draw.io	Entity Relationship Diagrams, use case diagrams, and activity diagrams.
Testing	Django Test Framework / pytest-django, Postman	Unit and integration testing of application logic; manual testing of API endpoints.
Environment management	Python venv, pip (requirements.txt)	Isolated, reproducible Python environment and dependency tracking.
Authentication & security	Django's built-in authentication framework	Session-based login with salted password hashing and role-based permissions.

Reference Materials

- Research journals and cybersecurity publications.
- CVE (Common Vulnerabilities and Exposures) documentation.
- CVSS (Common Vulnerability Scoring System) documentation.
- National Vulnerability Database (NVD) API documentation.
- Academic books on database systems, software engineering, and information security.

1.12 Project Plan

Activity	Weeks
Proposal Development	1–2
Literature Review	3–4
Requirements Analysis	5
Database Design	6
System Design	7
System Development	8–11
System Testing	12–13
Evaluation	14
Final Documentation	15
Final Presentation	16

The table above is presented visually below as a Gantt chart.

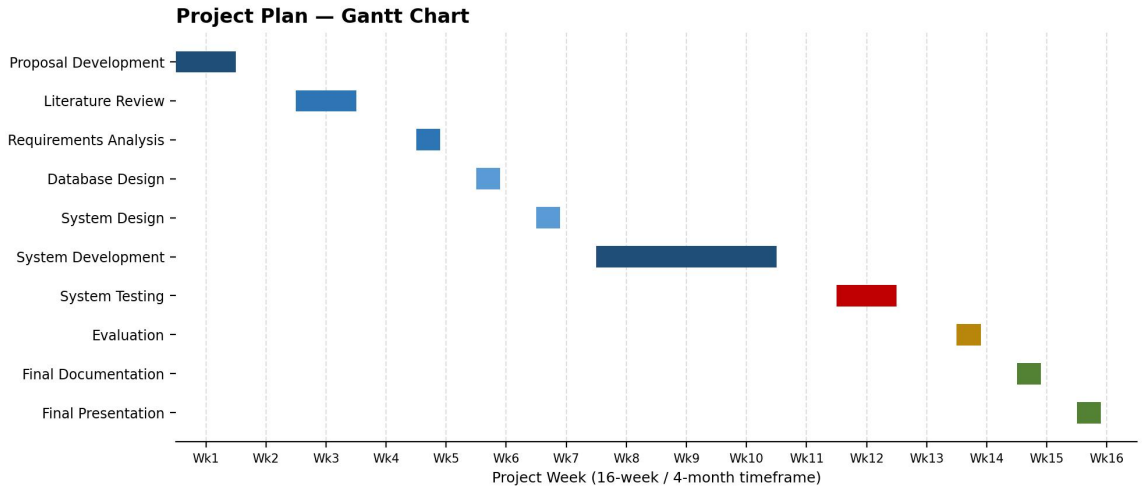


Fig 1. Gantt chart of the 16-week project plan (Chapter 1 proposal).

Chapter 2: First Review

This chapter builds on the project proposal presented in Chapter 1. It presents a review of literature that informed the project's design decisions, expands on the research methodology, restates the resource requirements as a confirmed technology stack, defines the functional and non-functional requirements of the system, presents the required modeling diagrams for a software project, and gives the project plan as a Gantt chart.

2.1 Literature Review

The Common Vulnerabilities and Exposures (CVE) database and the Common Vulnerability Scoring System (CVSS) form the technical foundation this project builds on. The CVE Program (n.d.) maintains a standardized, publicly available catalogue of disclosed vulnerabilities, each assigned a unique identifier, which allows disparate tools and organizations to refer to the same vulnerability unambiguously. FIRST (n.d.), the organization responsible for CVSS, defines it as a scoring framework that captures the principal technical characteristics of a vulnerability, its attack vector, complexity, privileges required, and impact on confidentiality, integrity, and availability, and expresses them as a severity score between 0.0 and 10.0. These two standards are the reason this project can build a credible prioritization system without having to invent its own vulnerability taxonomy from scratch.

However, CVSS was explicitly designed to measure technical severity, not organizational risk, and several sources converge on the same underlying critique. Recorded Future (2026) argues that relying

exclusively on CVSS leads organizations to spend disproportionate effort on vulnerabilities labelled 'critical' while losing the ability to prioritize the vulnerabilities that pose the greatest actual risk to a specific organization, since CVSS does not account for whether a vulnerability is being exploited in the wild or whether it affects a business-critical system. Shimizu and Hashimoto (2025) quantify this gap directly: their analysis found that CVSS-based prioritization alone requires security teams to address approximately 57% of all disclosed vulnerabilities to catch the ones that matter, while correctly identifying only around 20% of vulnerabilities that are actually exploited. This statistic is the clearest evidence available that CVSS-only prioritization is not simply imperfect but operationally unsustainable at scale, and it is the central justification for this project's Smart Priority Score, which treats CVSS as one input among several rather than the sole ranking criterion.

The industry has already moved toward combining CVSS with complementary signals rather than replacing it outright, and this project's design follows the same direction. Cloudsmith (2025) describes how the Exploit Prediction Scoring System (EPSS) and the CISA Known Exploited Vulnerabilities (KEV) catalogue are increasingly used alongside CVSS: EPSS estimates the probability that a vulnerability will be exploited in the near future, while KEV confirms vulnerabilities already observed being exploited, and combining all three signals allows an automated triage process to flag only what is genuinely urgent. Shimizu and Hashimoto (2025) built and empirically validated a decision-tree framework, Vulnerability Management Chaining, that formalizes exactly this kind of multi-signal combination and reports that it identifies dozens of actually-exploited vulnerabilities that neither EPSS nor KEV alone would have flagged. This project's Smart Priority Score follows the same philosophy of combining independent signals, but adds a factor these published approaches do not directly incorporate: organizational asset criticality, so that the same CVE affecting a critical finance server and a disposable test machine can be scored differently.

It is also important to review how existing, already-deployed systems approach this same problem, rather than situating this project only against academic literature. Cisco Vulnerability Management, formerly Kenna Security, is the most directly comparable existing system: it combines vulnerability data with asset priority using predictive modelling techniques including support-vector machines, random forest, and logistic regression, producing a Cisco Security Risk Score from 0 to 100 per vulnerability and an asset-level risk score from 0 to 1000 that explicitly factors in asset criticality, the same core idea underpinning this project's Smart Priority Score. This is strong independent validation that combining CVSS-type severity data with organizational asset criticality is a proven, industry-adopted approach, not merely a theoretical proposal. However, Cisco Vulnerability Management is a commercial, subscription-

licensed SaaS platform, which puts it out of reach for an organization of Kgori Insurance Brokers' size with no dedicated security budget, and is itself the gap this project's free, open-source implementation is intended to fill.

On the open-source side, DefectDojo, an OWASP Flagship project, is the most widely deployed free alternative: it aggregates and deduplicates vulnerability findings from over 200 security tools into a single source of truth, and, notably, is itself built on Python and Django, the same technology stack selected for this project in Section 1.11. DefectDojo demonstrates that this stack is a proven, viable choice for a vulnerability management platform at this scale. However, DefectDojo's core strength is aggregation, deduplication, and orchestration across tools, not predictive, asset-criticality-weighted risk scoring of the kind Cisco Vulnerability Management provides; it does not compute an equivalent of the Smart Priority Score out of the box. This project's position relative to both existing systems is therefore clear: it adopts Cisco Vulnerability Management's proven risk-scoring concept, but delivers it as a free, open-source, Django-based tool in the spirit of DefectDojo, rather than requiring either an enterprise licence or custom scoring logic bolted onto an aggregation-only platform.

On the remediation side of vulnerability management, Souppaya and Scarfone (2022), writing NIST's official guidance on enterprise patch management, frame patching as a resource-constrained decision problem: organizations rarely have the capacity to patch every known vulnerability immediately, and must weigh the security risk of delaying a patch against the operational risk of deploying it without adequate testing. This directly supports including patch availability and remediation progress as explicit factors in the Smart Priority Score, since a vulnerability with no available patch requires a different response, such as compensating controls, than one where a patch exists but has simply not yet been applied.

For the practical data backbone of the system, the National Vulnerability Database (n.d.) is the United States government's repository of CVE records enriched with CVSS scores, references, and metadata, made available through a public REST API. Using the live NVD API rather than manually entered vulnerability records was a deliberate design decision informed by this source: it means the system's Smart Priority Score is calculated against real, current vulnerability data rather than a static or fabricated dataset, which makes the testing and evaluation chapter's results considerably more meaningful.

On the database design underpinning the system, Elmasri and Navathe (2017) set out the principles of relational schema normalization, ensuring that vulnerability, asset, remediation, and user data are stored without redundancy and with referential integrity enforced through foreign key relationships. Given that

this system must reliably link a single vulnerability record to potentially many affected assets, and a single asset to many historical vulnerabilities, a properly normalized schema, rather than a flat or denormalized one, is necessary to avoid data anomalies as the volume of imported CVE records grows.

On the broader question of why asset criticality belongs in a risk score at all, Stallings (2021) frames risk assessment as fundamentally about evaluating both the likelihood of a threat and the impact it would have on a specific asset, not the threat in isolation. This is the conceptual basis for weighting the Smart Priority Score by asset criticality: two vulnerabilities with identical CVSS scores represent different levels of actual risk if one affects a system with low business impact and the other affects a system that is critical to organizational operations.

Finally, on the development approach itself, Pressman and Maxim (2020) describe iterative software engineering practice as building and validating a system in successive, working increments rather than attempting a single complete implementation, which reduces the risk of discovering a fundamental design flaw late in a project. This directly supports the Design Science Research combined with Iterative SDLC approach adopted for this project, in which the database schema, the NVD integration, the scoring engine, and the dashboard are each built and verified before the next component is layered on top.

2.2 Research Methodology

The development life cycle model adopted for this project is the Iterative Model, combined with Design Science Research as the overarching research paradigm. Design Science Research is appropriate here because the project's primary contribution is a working artefact, the Smart Priority Score engine and the system built around it, evaluated against realistic scenarios, rather than a purely theoretical finding. Within that paradigm, the Iterative Model governs how the artefact itself is built: the database schema is designed and validated first, since every later component depends on it; the NVD API integration and Smart Priority Score calculation logic are built and tested against real imported CVE data next; and the dashboard and reporting layer are built last, once there is real scored data to display. Each cycle's output is tested before the next cycle begins, rather than designing the complete system up front in a single Waterfall-style pass.

On data gathering: this project has no human research participants, since it is a technical system built and evaluated against publicly available vulnerability data and fabricated organizational scenarios rather than a study of user behaviour. The primary data-gathering method is document analysis, requirements and design decisions are derived from published standards (CVE, CVSS), NIST guidance on patch

management, and the academic and industry literature reviewed in Section 2.1, rather than from interviews or surveys. The secondary data source is the NVD API itself, which supplies the real vulnerability records the system is built and tested against.

The finished system will be evaluated experimentally rather than through survey methods. Sample organizational scenarios, each with a fabricated asset inventory of varying criticality, will be run against real CVE/CVSS data imported from the NVD API, and the resulting Smart Priority Score rankings will be compared against a naive CVSS-only ranking of the same vulnerabilities to demonstrate concretely where and how organizational context changes prioritization outcomes.

2.3 Resource Requirements

The technology stack confirmed in Section 1.11 remains the resource plan for implementation. Python 3.11+ with Django 5 was selected over alternatives such as Java/Spring Boot or C#/.NET because Django's built-in authentication, ORM, and admin scaffolding substantially reduce the amount of boilerplate security-sensitive code, such as password handling and session management, that would otherwise need to be written and tested from scratch within a 16-week solo project. MySQL 8.0 was selected over Microsoft SQL Server for its zero licensing cost and native compatibility with Django's ORM.

No additional hardware procurement is required beyond a standard development laptop or desktop, since the entire system, application server, database, and development environment, runs locally during development. The only external dependency is network access to the NVD API, which is free but rate-limited for unauthenticated requests; an NVD API key will be requested ahead of the development phase to avoid rate-limit interruptions once the system begins importing larger volumes of CVE data.

2.4 Functional Requirements / Non-Functional Requirements

The following functional requirements apply to this system:

- The system shall require administrators to authenticate via a secure login before accessing any vulnerability, asset, or remediation data.
- The system shall import vulnerability records, including CVE ID, description, and CVSS base score, from the NVD API on a scheduled or on-demand basis.
- The system shall allow administrators to maintain an asset inventory, including a criticality rating for each asset.

- The system shall match imported vulnerability records to the assets they affect.
- The system shall calculate a Smart Priority Score for each vulnerability using CVSS score, asset criticality, exploit availability, patch availability, and remediation status.
- The system shall classify each vulnerability into a risk level (Critical, High, Medium, or Low) based on its Smart Priority Score.
- The system shall display prioritized vulnerabilities on a dashboard ordered by Smart Priority Score.
- The system shall allow administrators to assign remediation tasks to responsible users and track remediation status through to closure.
- The system shall maintain an audit log of all changes made to vulnerability, asset, and remediation records.
- The system shall generate reports summarizing vulnerability status and remediation progress.

2.4.1 Non-Functional Requirements

Requirement: The vulnerability dashboard should remain responsive enough to be usable during a live administrative review session, even as the number of stored vulnerability records grows.

Fit Criterion: The dashboard should render a prioritized list of up to 5,000 vulnerability records in under 5 seconds on the development hardware described in Section 1.11.

Requirement: User credentials must never be recoverable in plain text, to protect administrator accounts if the database is ever compromised.

Fit Criterion: 100% of stored passwords must be salted and hashed using Django's default password hashing algorithm, verified by direct inspection of the user table showing no plaintext values.

Requirement: The Smart Priority Score calculation should be consistent and repeatable, so that administrators can trust the ranking is not arbitrary.

Fit Criterion: Given the same vulnerability, asset, and remediation data, the system should return an identical Smart Priority Score on every calculation, verified by re-running the scoring engine against a fixed test dataset at least three times.

2.5 Modeling Diagrams

As this is a software development project, a Use Case diagram is provided, showing the Security Administrator and the external NVD API system interacting with the core functions of the system, together with an Activity diagram detailing the system's central process: calculation of the Smart Priority Score.

2.5.1 Use Case Diagram

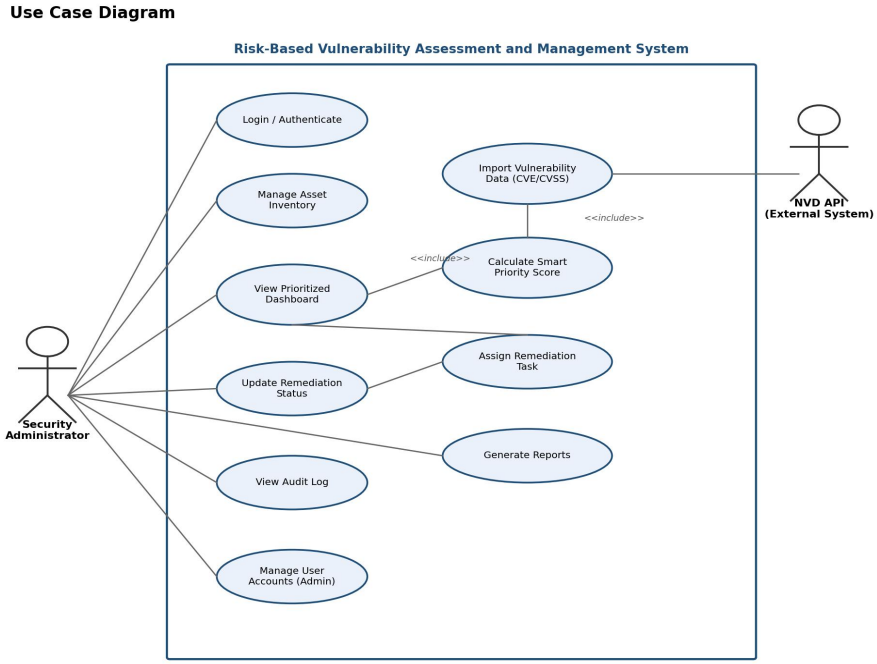


Fig 2. Use case diagram for the Risk-Based Vulnerability Assessment and Management System.

2.5.2 Activity Diagram

The activity diagram below details the Smart Priority Score calculation process, the system's core function, from vulnerability import through to dashboard update.

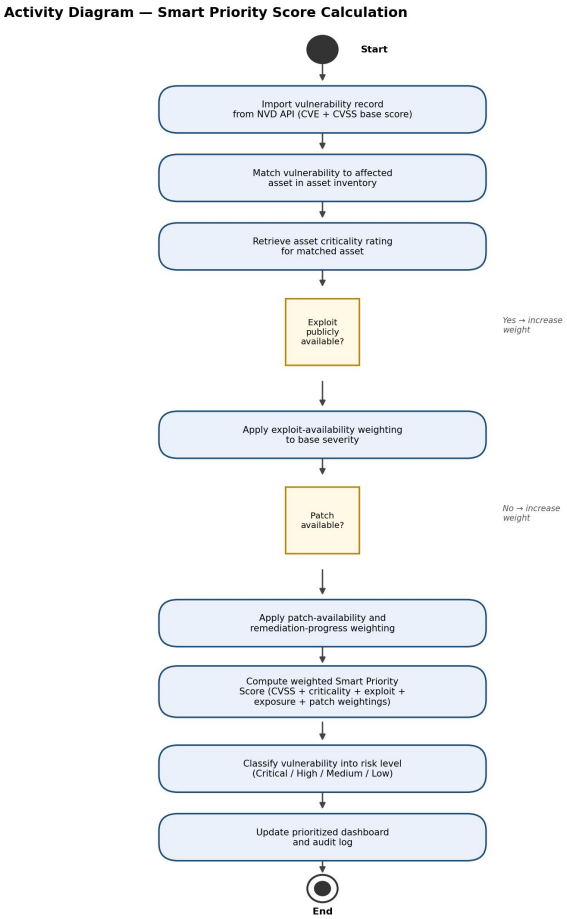


Fig 3. Activity diagram for Smart Priority Score calculation.

2.6 Project Plan

The 16-week project plan from Section 1.12 is presented below as a Gantt chart.

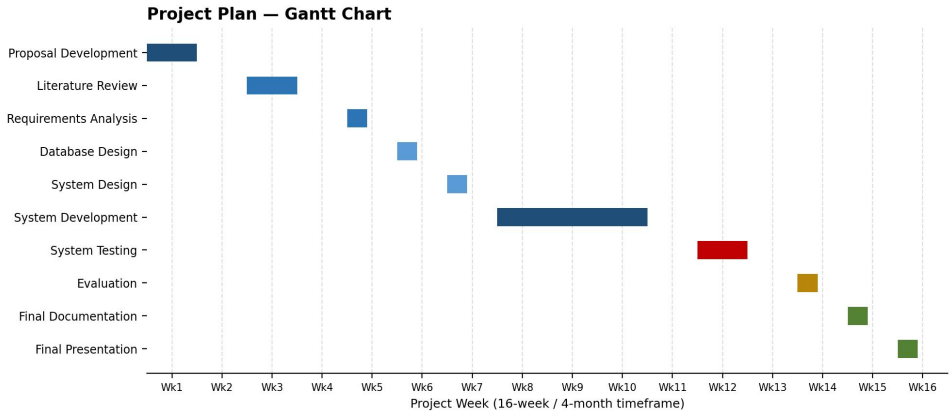


Fig 4. Gantt chart of the 16-week project plan.

Chapter 3: Second Review

This chapter presents the Second Review deliverables for this software development project, building on the proposal in Chapter 1 and the literature review and requirements established in Chapter 2. As this is a software project, the module guidelines call for a Class diagram, Object diagram, Sequence diagram, Communication diagram, State chart, and Process Design using pre- and post-conditions, each shown below for the system's core scenario: calculating a vulnerability's Smart Priority Score.

3.1 UML Design Guidelines

The diagrams in this chapter were developed starting from the Class diagram, since it defines the structural vocabulary, the classes, attributes, and associations, that every other diagram in this chapter depends on. Each subsequent diagram was then built to illustrate the same central scenario from a different angle: the Object diagram shows a concrete snapshot of these classes populated with real values; the Sequence and Communication diagrams show the same interaction between objects from two different notations; the State chart shows how a single object's state changes over its lifecycle; and the Process Design section defines the formal pre- and post-conditions for the scenario's key operation.

3.2 Class Diagram

The classes below were identified directly from the system's proposed solution in Chapter 1: User represents the role of the security administrator who operates the system; Asset and Vulnerability represent the two tangible data sources the system combines; AssetVulnerability is an association class linking a specific asset to a specific vulnerability, and is where the Smart Priority Score itself is calculated and stored, since the score is a property of that particular pairing, not of the asset or vulnerability alone; RemediationTask represents the event of assigning and tracking remediation work; and AuditLog represents the event of recording administrator actions for accountability.

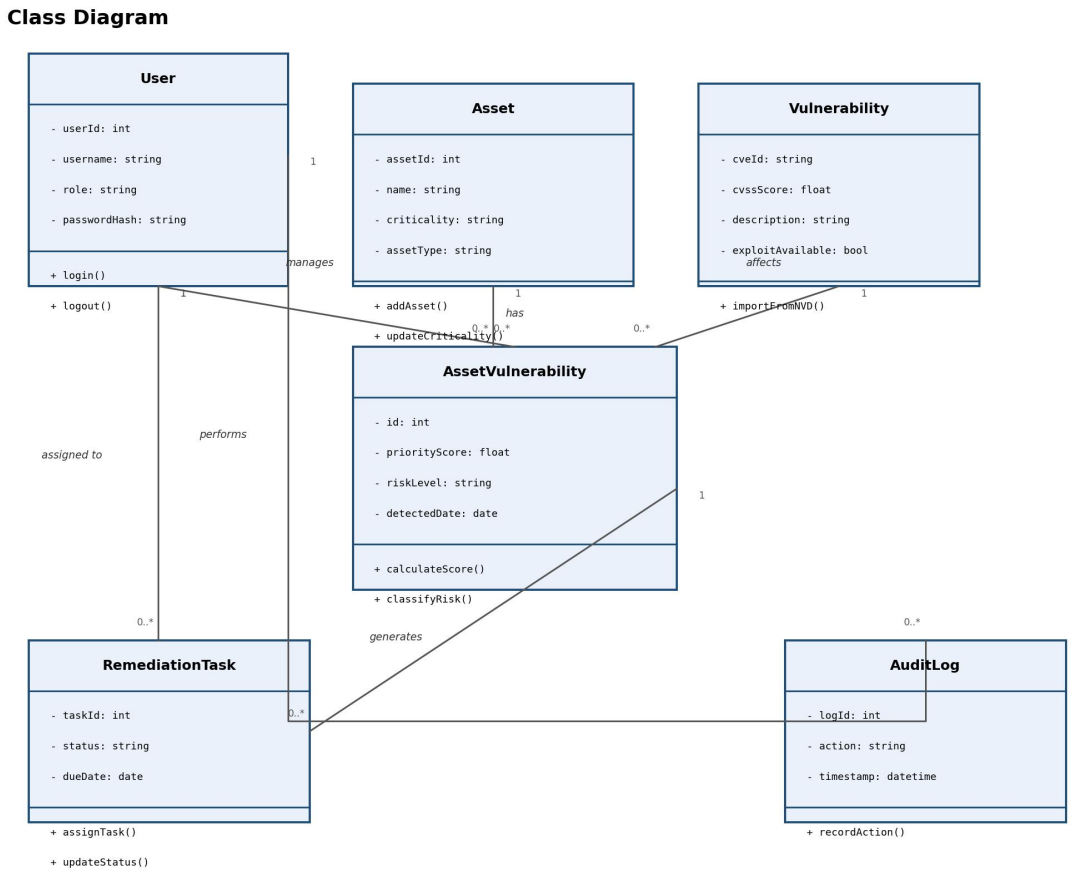


Fig 5. Class diagram for the Risk-Based Vulnerability Assessment and Management System.

Multiplicities reflect the real relationships in the domain: one Asset can have many AssetVulnerability records over time (1 to 0..*), one Vulnerability can likewise affect many assets (1 to 0..*), and a single AssetVulnerability can generate zero or more RemediationTask records as work is assigned and reassigned (1 to 0..*).

3.3 Object Diagrams

The object diagram below shows the Class diagram's state at one particular instant: immediately after administrator admin1 has triggered a Smart Priority Score calculation for a specific critical finance server against a newly imported, high-severity CVE, and the resulting AssetVulnerability record has been scored and has generated an open remediation task.

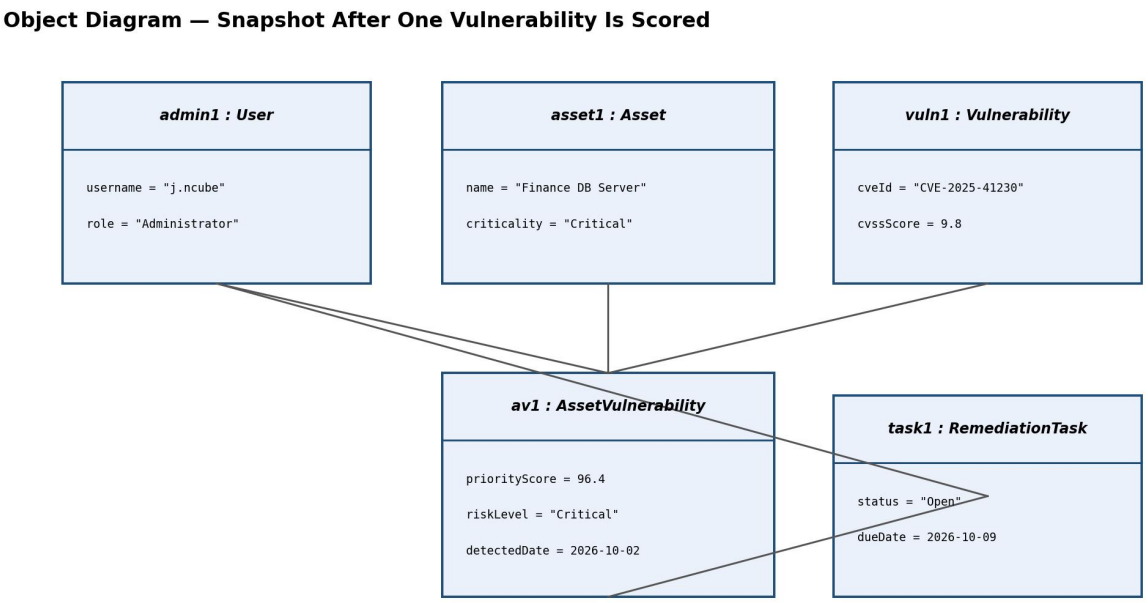


Fig 6. Object diagram showing a populated instance after one vulnerability has been scored.

3.4 Sequence Diagram

The sequence diagram below captures a single scenario, an administrator triggering the calculation of a Smart Priority Score, consistent with the guidance that a sequence diagram should represent one scenario rather than several combined. The AssetVulnerability object (av1) initiates two outgoing calls to collect the data it needs, to Vulnerability for the CVSS score and to Asset for the criticality rating, shown as overlaid activation bars since these are self-initiated calls made while av1 is still processing the original request. The three internal calculation steps, computeWeightedScore(), classifyRiskLevel(), and setPriorityScore(), are shown as self-messages on av1's own lifeline.

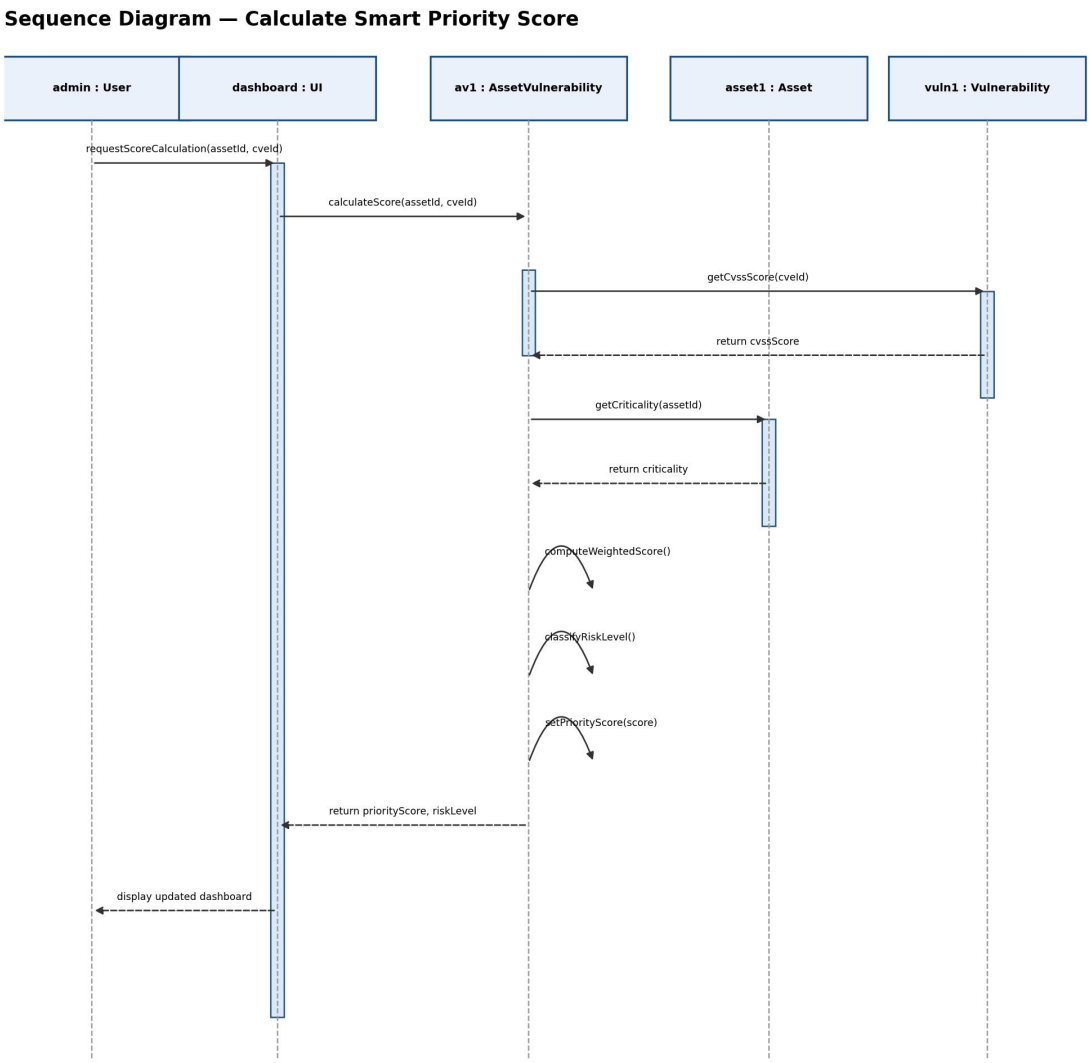


Fig 7. Sequence diagram for Smart Priority Score calculation.

3.5 Communication Diagram

The communication diagram below represents the identical scenario shown in the sequence diagram above, but numbers each message instead of ordering them vertically by time. Message 1 initiates the request; message 2 and its sub-messages (2.1 to 2.4) represent the calculation logic carried out by av1, following the standard communication-diagram numbering convention of decimal sub-numbers for messages sent as part of handling an earlier message.

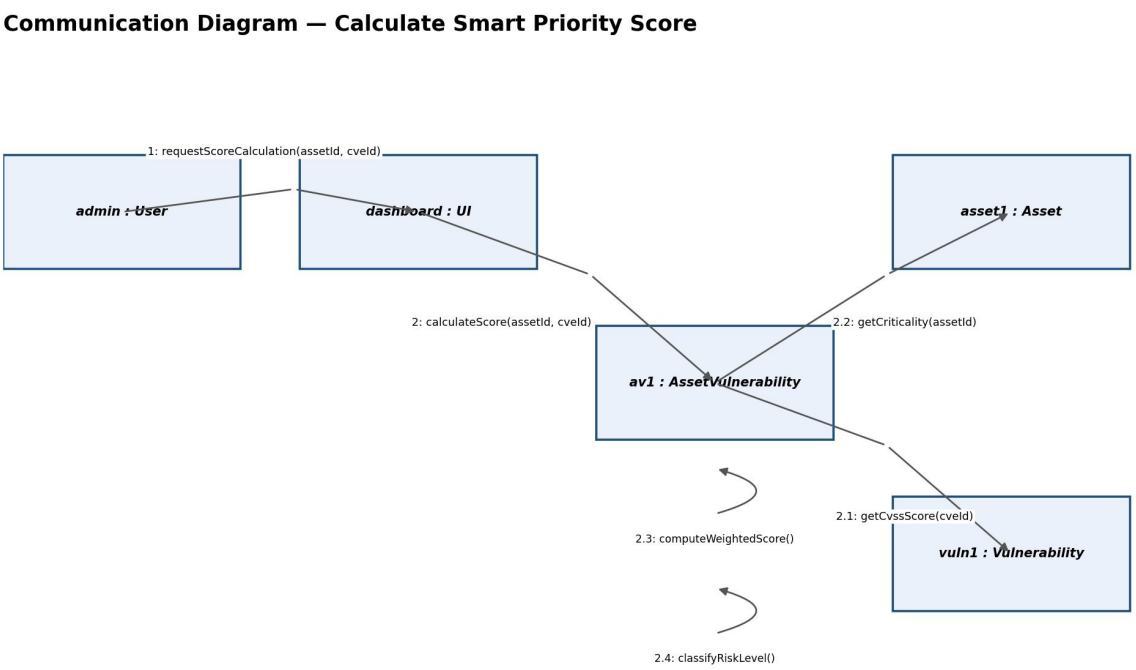


Fig 8. Communication diagram for Smart Priority Score calculation.

3.6 State Charts

The state chart below shows how a single Asset Vulnerability record's remediation status changes over its lifecycle, from the moment a vulnerability is first detected against an asset through to closure. A vulnerability can be closed by two different routes, either remediated through to a verified fix, or formally risk-accepted by an administrator who judges the fix is not currently warranted, which the state chart represents as two distinct paths to the End state rather than forcing every vulnerability through remediation.

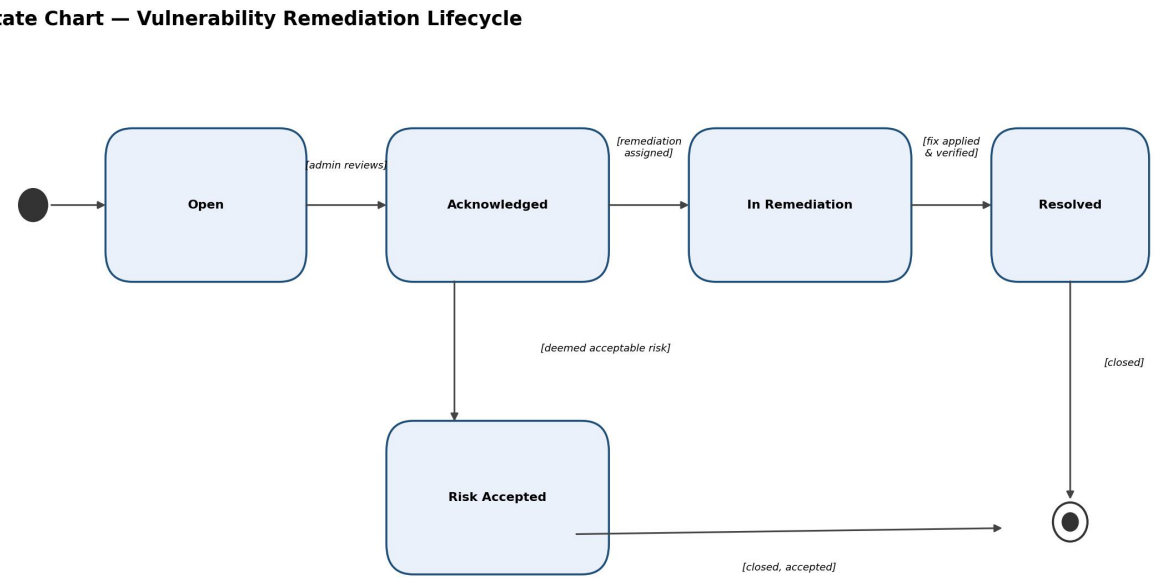


Fig 9. State chart for the vulnerability remediation lifecycle.

3.7 Process Design

The operation chosen for formal pre- and post-condition specification is calculateScore, the core operation of the AssetVulnerability class shown in the Class diagram above, since this is the single operation every other diagram in this chapter is built around.

```
Context AssetVulnerability::calculateScore(asset: Asset, vulnerability:
Vulnerability): float
```

Pre

```
-- asset is a registered Asset in the system
-- vulnerability is a Vulnerability record already imported from the NVD API
-- asset.criticality has been set (not null)
-- vulnerability.cvssScore has been set (not null)
```

Post

```
-- a new AssetVulnerability object will have been created, linking asset and
vulnerability
-- AssetVulnerability.priorityScore will have been set to a value between 0
and 100
```

```
-- AssetVulnerability.riskLevel will have been set to one of {Low, Medium, High, Critical}, consistent with priorityScore
-- if riskLevel is Medium, High, or Critical, a new RemediationTask will have been created with status "Open"
-- a new AuditLog entry will have been created recording the calculation
```

Chapter 4: Final Project

This chapter presents the final project: the Risk-Based Vulnerability Assessment and Management System as it was actually built and run. Section 4.1 describes the implementation in the order it was built, using screenshots of the running system, and states where it differs from the design in Chapters 1 to 3. Section 4.2 describes the black-box testing of the system, with a screenshot for every test case. Section 4.3 evaluates the project against the objectives set in Chapter 1. Every screenshot in this chapter was taken from the running application connected to its MySQL database; none are mock-ups. In line with Section 1.8, all assets, users and remediation activity are fabricated, and all vulnerability records are real CVE records imported from the National Vulnerability Database, except the two clearly labelled CVE-TEST records and the synthetic records used for the performance test in Section 4.2.

4.1 Implementation of the design

Following the iterative approach chosen in Section 2.2, the system was built and checked one layer at a time, each layer depending on the one before it: the development environment, the database, the NVD integration, the Smart Priority Score engine, the asset inventory and dashboard, the remediation workflow, and finally reporting, the evaluation view and auditing. The sections below follow that order.

4.1.1 Development environment

The technology stack is the one confirmed in Section 1.11: Python with the Django web framework, a MySQL 8.0 database, Bootstrap 5 and Chart.js for the user interface, and the NVD REST API 2.0 as the vulnerability data source. The installed versions are Python 3.12.10, Django 5.2.18 and MySQL 8.0.45 Community Server, running as a Windows service. Dependencies are isolated in a Python virtual environment and listed in requirements.txt, and the source code is under Git version control and hosted on GitHub. Fig 10 shows the environment being verified: Django finds no configuration issues, the database migration is applied, the application is connected to the rbvms database as the rbvms user, and the development server starts.



Fig 10. Development environment verified: Python and Django versions, system check, applied migration, MySQL 8.0.45 connection and the development server starting.

4.1.2 Database

The database was built first, as every later component depends on it. Each class in the Class diagram (Fig 5) became a table: User, Asset, Vulnerability, AssetVulnerability, RemediationTask and AuditLog. The many-to-many relationship between assets and vulnerabilities is resolved through the AssetVulnerability table, which holds the Smart Priority Score, risk level, lifecycle status and detection date of one vulnerability on one asset, exactly as the Class diagram describes. Fig 11 shows the entity-relationship diagram generated directly from the foreign keys of the live MySQL database, and Fig 12 shows the columns, primary keys, six foreign keys and unique constraints read from MySQL's information_schema. The unique constraints prevent duplicate assets, duplicate CVE records and the same vulnerability being recorded twice against one asset, and the foreign keys enforce referential integrity as recommended by Elmasri and Navathe (2017).

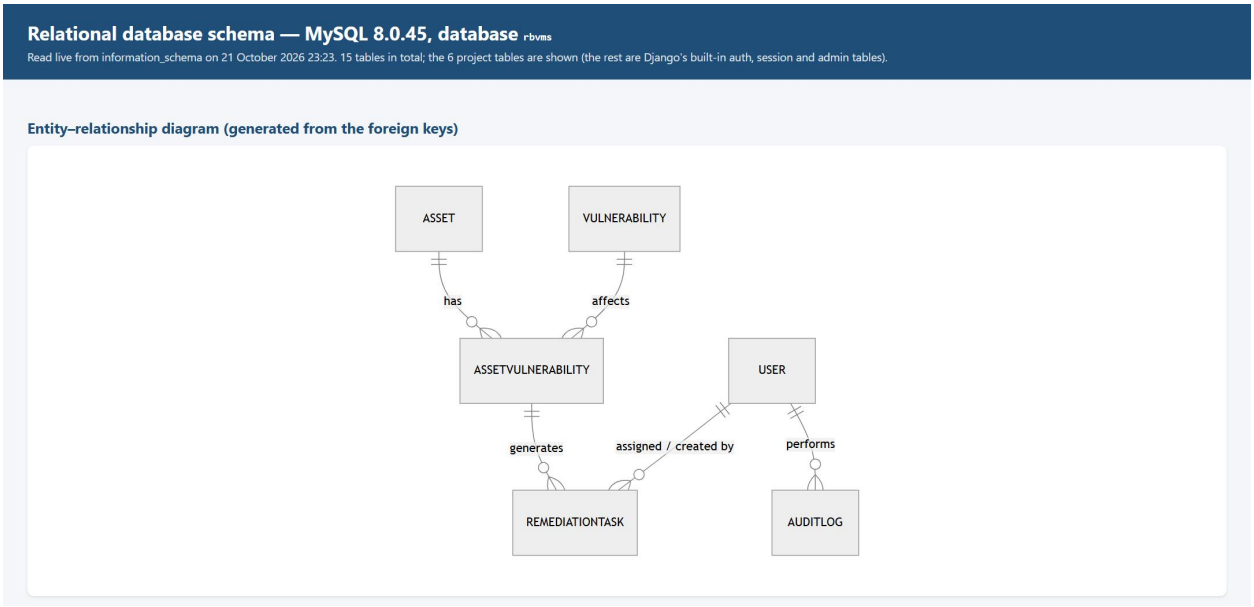


Fig 11. Entity-relationship diagram generated from the foreign keys of the live MySQL 8.0.45 database.

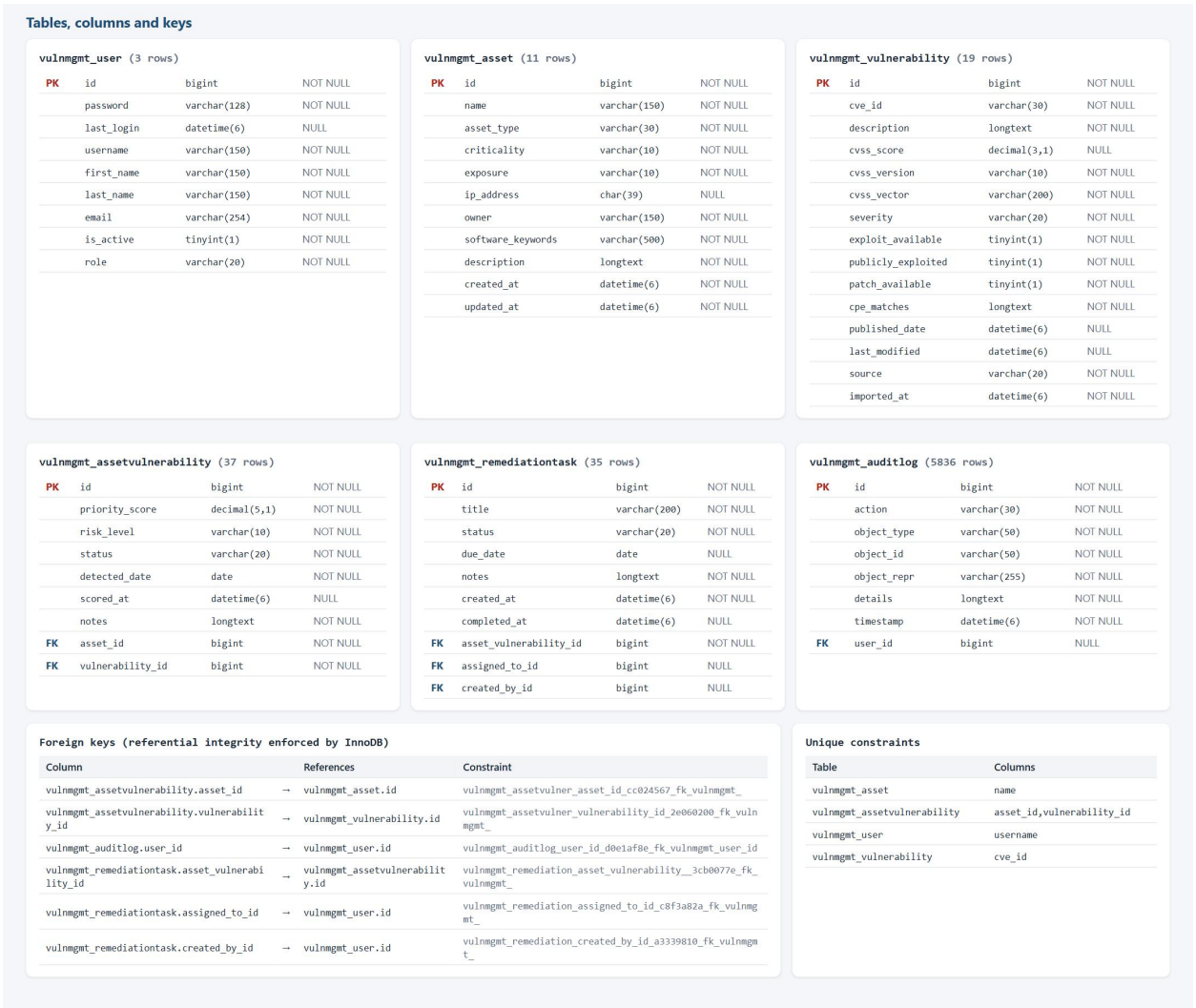


Fig 12. Project tables with their columns, primary keys (PK), foreign keys (FK) and unique constraints, read from MySQL information_schema.

4.1.3 Vulnerability data from the NVD

The second layer imports vulnerability records from the NVD CVE API 2.0, by CVE ID, keyword, publication date range or the CISA Known Exploited Vulnerabilities (KEV) list. For each CVE the system stores the description, the CVSS base score and vector (preferring the NVD's own CVSS v3.1 score), whether a public exploit or a patch is referenced, whether the CVE is in the KEV catalogue, and the list of affected products as CPE names. Each imported CVE is then matched against the asset inventory: an asset is affected when one of its software keywords appears in the CVE's CPE list. Matched findings are scored immediately. The client respects the NVD rate limits and, because the NVD regularly took over a minute to answer during development, waits up to two minutes per request and makes up to three attempts before reporting an error. Fig 13 shows a live import of CVE-2021-45105, which created one record and three scored findings, and Fig 14 shows an imported record with its CVSS vector, affected products and the score on each affected asset.

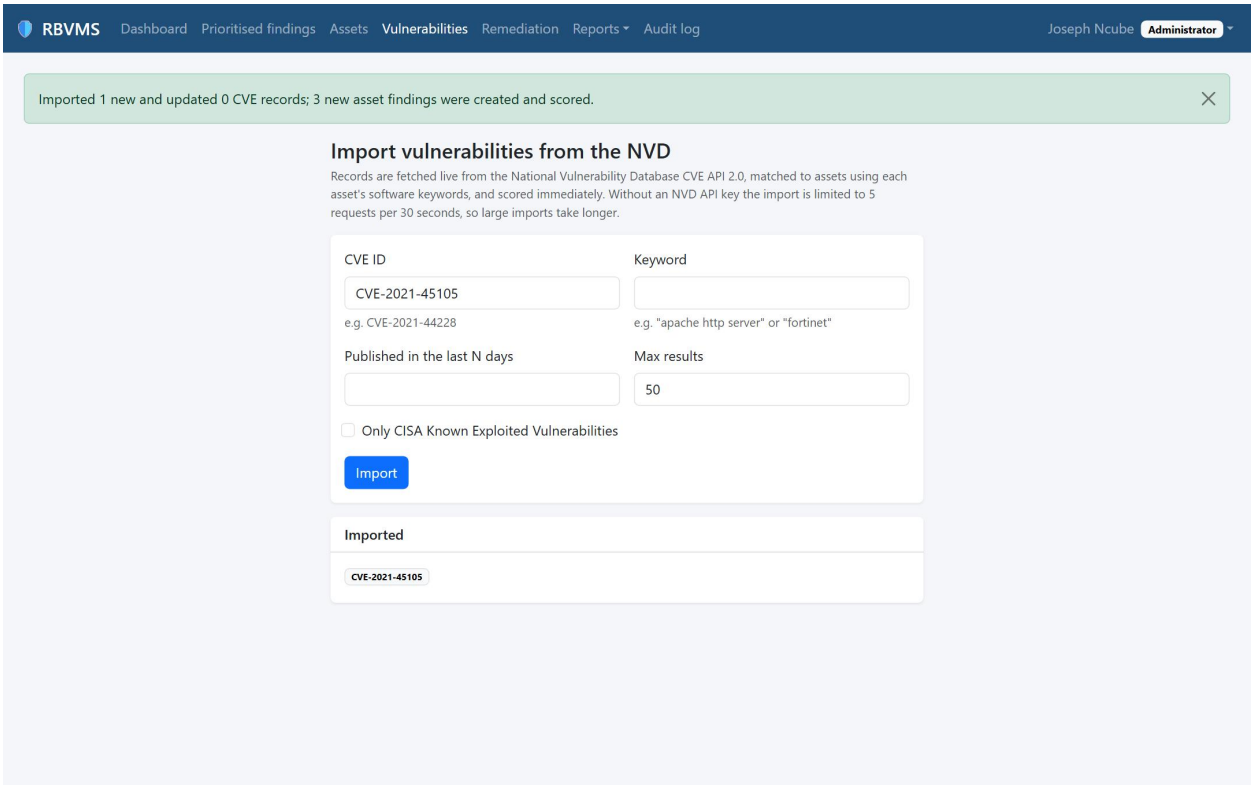


Fig 13. Live import of CVE-2021-45105 from the NVD CVE API 2.0: one new record, matched to and scored on three assets.

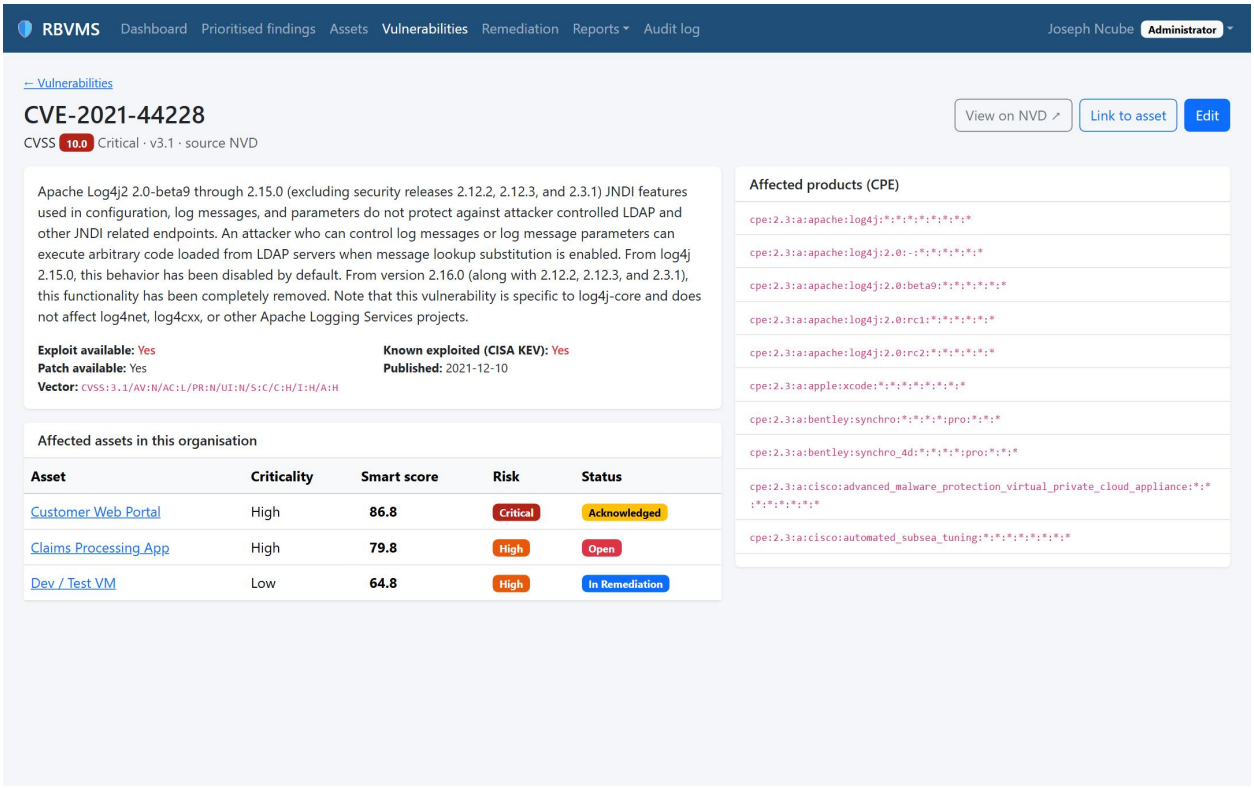


Fig 14. Imported record for CVE-2021-44228 (Log4Shell): NVD description, CVSS v3.1 score and vector, affected products (CPE), and the Smart Priority Score on each affected asset.

4.1.4 Smart Priority Score engine

The scoring engine implements the factors listed in Section 1.6. Each factor is converted to a value between 0 and 1 and multiplied by a fixed weight; the weights add up to 100, so the score runs from 0 to 100 (Table 1). The score is then classified into a risk level: Critical at 80 or above, High at 60, Medium at 40, and Low below that. As specified in Section 3.7, calculating a score also sets the risk level, creates an Open remediation task for Medium, High and Critical findings (due 30, 14 or 7 days after detection), and records the calculation in the audit log. Scores are recalculated automatically whenever one of their inputs changes, for example when an asset's criticality is edited or a finding moves through its lifecycle.

Table 1. Smart Priority Score factors and weights as implemented.

Factor	Weight	Value used

Factor	Weight	Value used
CVSS base score	30	Score divided by 10
Asset criticality	25	Low 0.25, Medium 0.5, High 0.75, Critical 1.0
Exploit available	10	1 if a public exploit is referenced, otherwise 0
Actively exploited (CISA KEV)	10	1 if listed in the KEV catalogue, otherwise 0
Exposure	10	Internal 0.3, DMZ 0.7, Internet-facing 1.0
Affected systems	5	Active assets with the same CVE divided by 5, capped at 1
Patch unavailable	5	1 if no patch or advisory is referenced, otherwise 0
Remediation pending	5	Open or Acknowledged 1.0, In Remediation 0.5, closed states 0

Fig 15 shows the scoring model as presented to users, and Fig 16 shows the breakdown for one finding. CVE-2020-1472 (Zerologon) has an NVD CVSS score of only 5.5 (Medium), but on the Critical Finance DB Server, where it is known to be actively exploited, its Smart Priority Score is 71.5 (High). This is the gap identified in Section 1.5: a CVSS-only ranking would have treated it as a medium-priority issue.

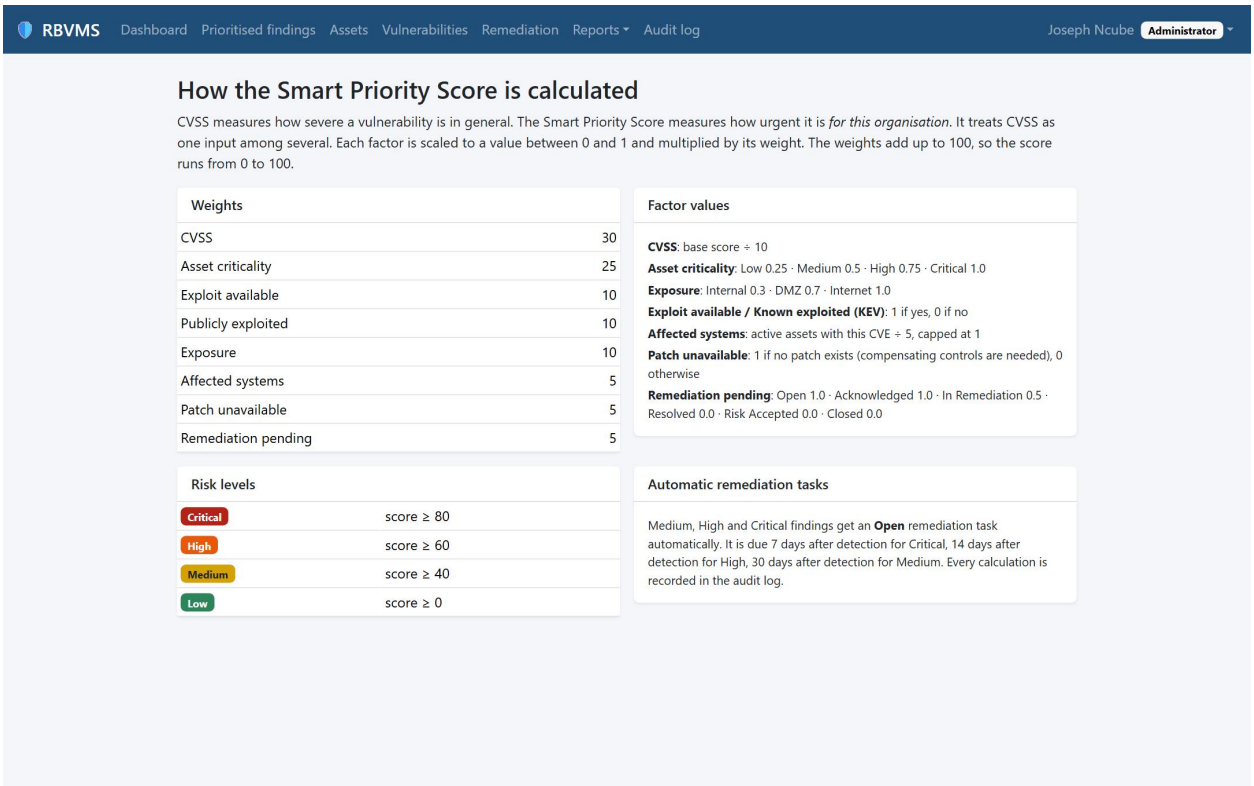


Fig 15. Smart Priority Score model shown to users: weights, factor values, risk bands and automatic remediation deadlines.

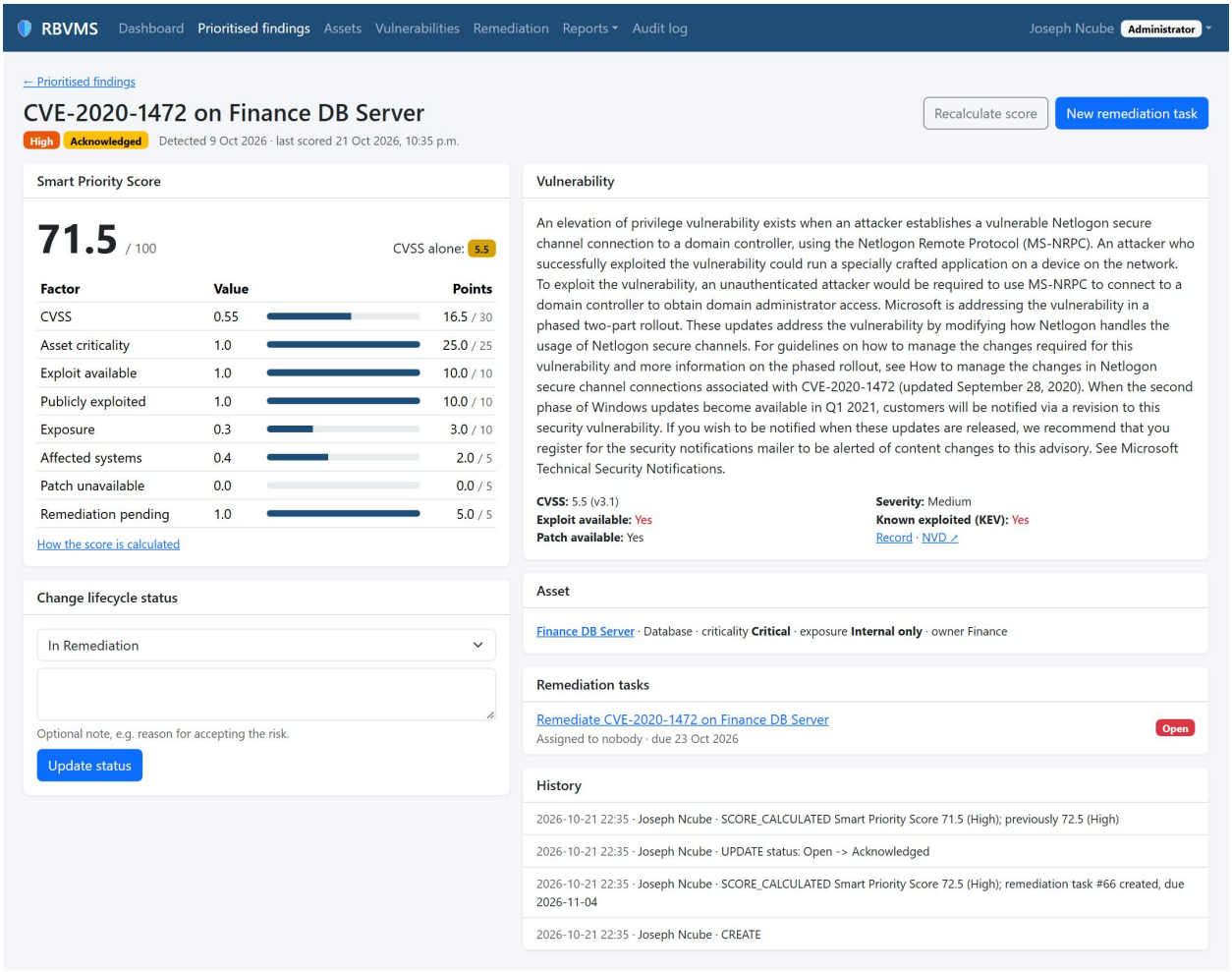


Fig 16. Score breakdown for CVE-2020-1472 (Zerologon) on the Finance DB Server: CVSS 5.5, Smart Priority Score 71.5 (High).

4.1.5 Asset inventory and dashboard

Administrators maintain the asset inventory (Fig 17), recording each asset's type, business criticality, exposure, owner and software keywords. The dashboard (Fig 18) summarizes the current position for a security administrator: counts of assets, tracked CVEs, active findings, critical findings, open and overdue tasks; charts of active findings by risk level, findings by lifecycle status and the riskiest assets; the ten highest-priority findings; the signed-in user's own tasks; and recent activity. The full prioritized list (Fig 19) can be filtered by text, risk level, status and asset.

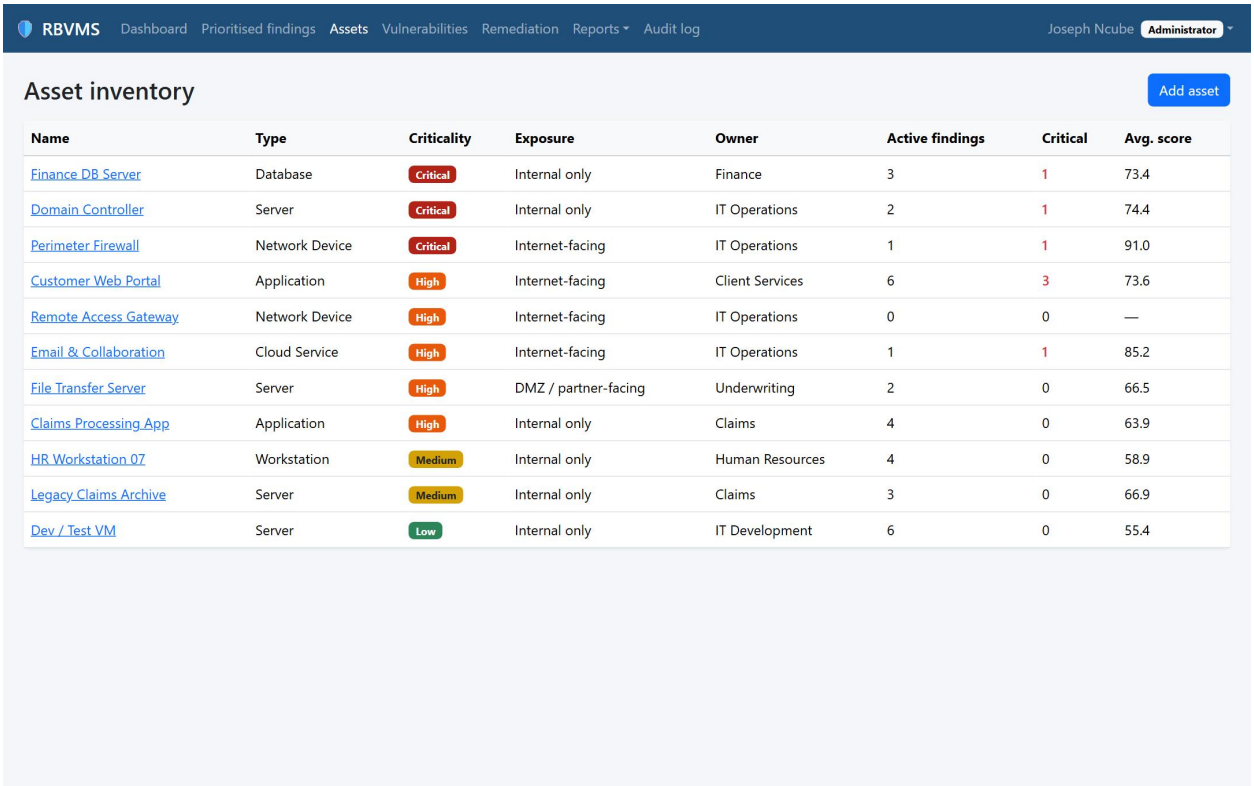


Fig 17. Asset inventory with criticality, exposure, owner and active findings per asset.

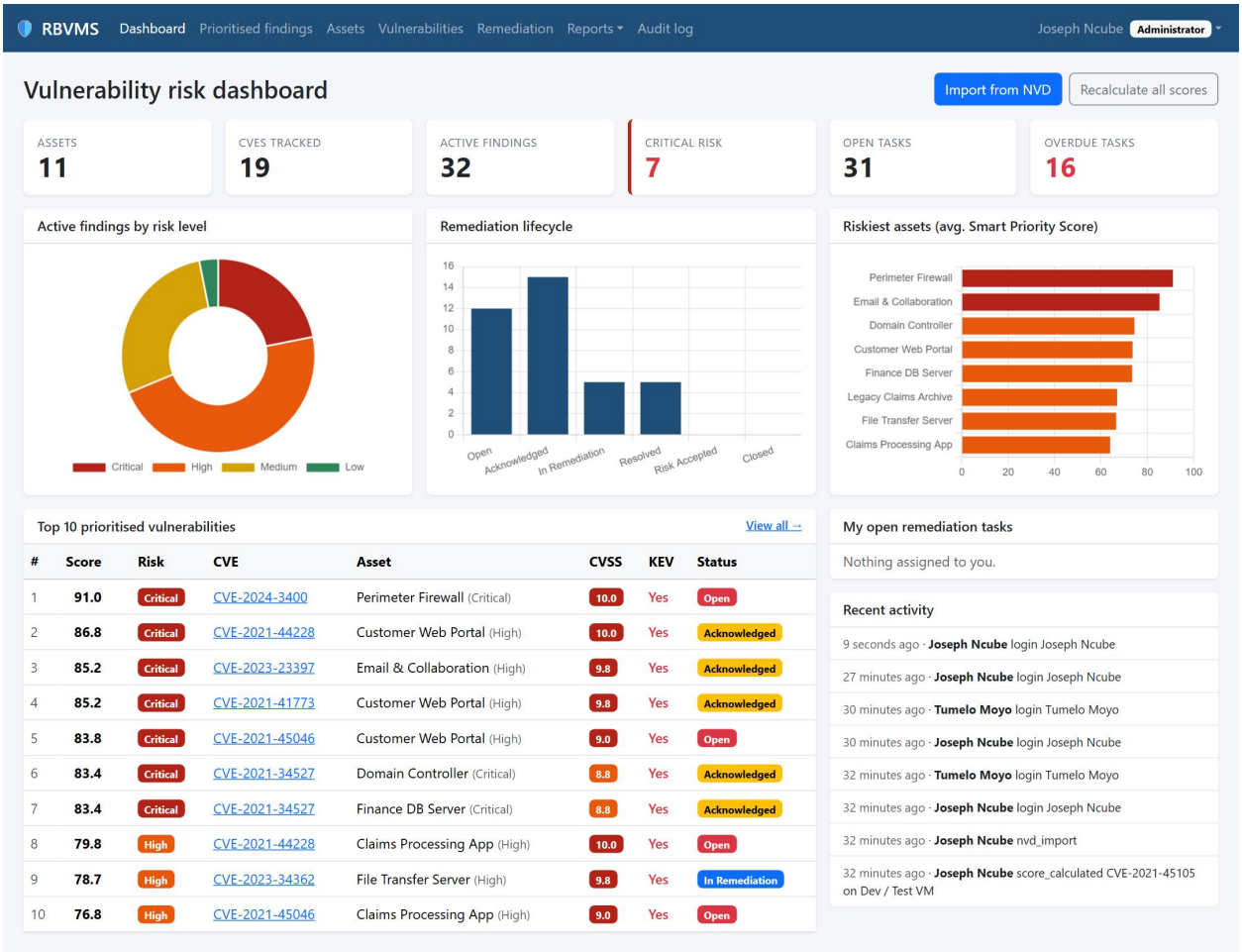


Fig 18. Vulnerability risk dashboard for a security administrator.

RBVMS Dashboard Prioritised findings Assets Vulnerabilities Remediation Reports Audit log Joseph Ncube Administrator

Prioritised findings 32 matching

Link vulnerability to asset

Search: CVE, asset or text | Risk level: All | Status: Active (not closed) | Asset: All | Per page: 50 | Filter

Rank	Smart score	Risk	CVE	Asset	Criticality	Exposure	CVSS	Exploit	KEV	Patch	Status	Detected
1	91.0	Critical	CVE-2024-3400	Perimeter Firewall	Critical	Internet	10.0	Yes	Yes	Yes	Open	3 Oct 2026
2	86.8	Critical	CVE-2021-44228	Customer Web Portal	High	Internet	10.0	Yes	Yes	Yes	Acknowledged	18 Oct 2026
3	85.2	Critical	CVE-2023-23397	Email & Collaboration	High	Internet	9.8	Yes	Yes	Yes	Acknowledged	21 Sep 2026
4	85.2	Critical	CVE-2021-41773	Customer Web Portal	High	Internet	9.8	Yes	Yes	Yes	Acknowledged	22 Sep 2026
5	83.8	Critical	CVE-2021-45046	Customer Web Portal	High	Internet	9.0	Yes	Yes	Yes	Open	21 Oct 2026
6	83.4	Critical	CVE-2021-34527	Domain Controller	Critical	Internal	8.8	Yes	Yes	Yes	Acknowledged	13 Oct 2026
7	83.4	Critical	CVE-2021-34527	Finance DB Server	Critical	Internal	8.8	Yes	Yes	Yes	Acknowledged	10 Oct 2026
8	79.8	High	CVE-2021-44228	Claims Processing App	High	Internal	10.0	Yes	Yes	Yes	Open	21 Oct 2026
9	78.7	High	CVE-2023-34362	File Transfer Server	High	DMZ	9.8	Yes	Yes	Yes	In Remediation	27 Sep 2026
10	76.8	High	CVE-2021-45046	Claims Processing App	High	Internal	9.0	Yes	Yes	Yes	Open	21 Oct 2026
11	71.9	High	CVE-2023-23397	HR Workstation 07	Medium	Internal	9.8	Yes	Yes	Yes	Acknowledged	18 Sep 2026
12	71.5	High	CVE-2020-1472	Finance DB Server	Critical	Internal	5.5	Yes	Yes	Yes	Acknowledged	9 Oct 2026
13	70.9	High	CVE-2019-0708	Legacy Claims Archive	Medium	Internal	9.8	Yes	Yes	Yes	Acknowledged	30 Sep 2026
14	70.9	High	CVE-2021-34527	HR Workstation 07	Medium	Internal	8.8	Yes	Yes	Yes	Open	7 Oct 2026
15	70.9	High	CVE-2021-34527	Legacy Claims Archive	Medium	Internal	8.8	Yes	Yes	Yes	Acknowledged	4 Oct 2026
16	67.6	High	CVE-2024-6387	Customer Web Portal	High	Internet	8.1	Yes	No	Yes	In Remediation	1 Oct 2026
17	65.7	High	CVE-2021-41773	Dev / Test VM	Low	Internal	9.8	Yes	Yes	Yes	Open	19 Sep 2026
18	65.4	High	CVE-2023-38545	Domain Controller	Critical	Internal	9.8	No	No	Yes	Open	15 Sep 2026
19	65.4	High	CVE-2023-38545	Finance DB Server	Critical	Internal	9.8	No	No	Yes	Acknowledged	12 Sep 2026

Fig 19. Prioritized findings list ordered by Smart Priority Score, with filters.

4.1.6 Remediation workflow

Remediation follows the state chart in Fig 9. A finding moves from Open to Acknowledged, then either to In Remediation and Resolved, or to Risk Accepted, before being Closed; the interface only offers transitions the state chart allows. Assigning a remediation task to a user moves an acknowledged finding into remediation automatically. Fig 20 shows the task list with assignees and overdue tasks, and Fig 21 shows a finding in remediation with its task and the history of status and score changes.

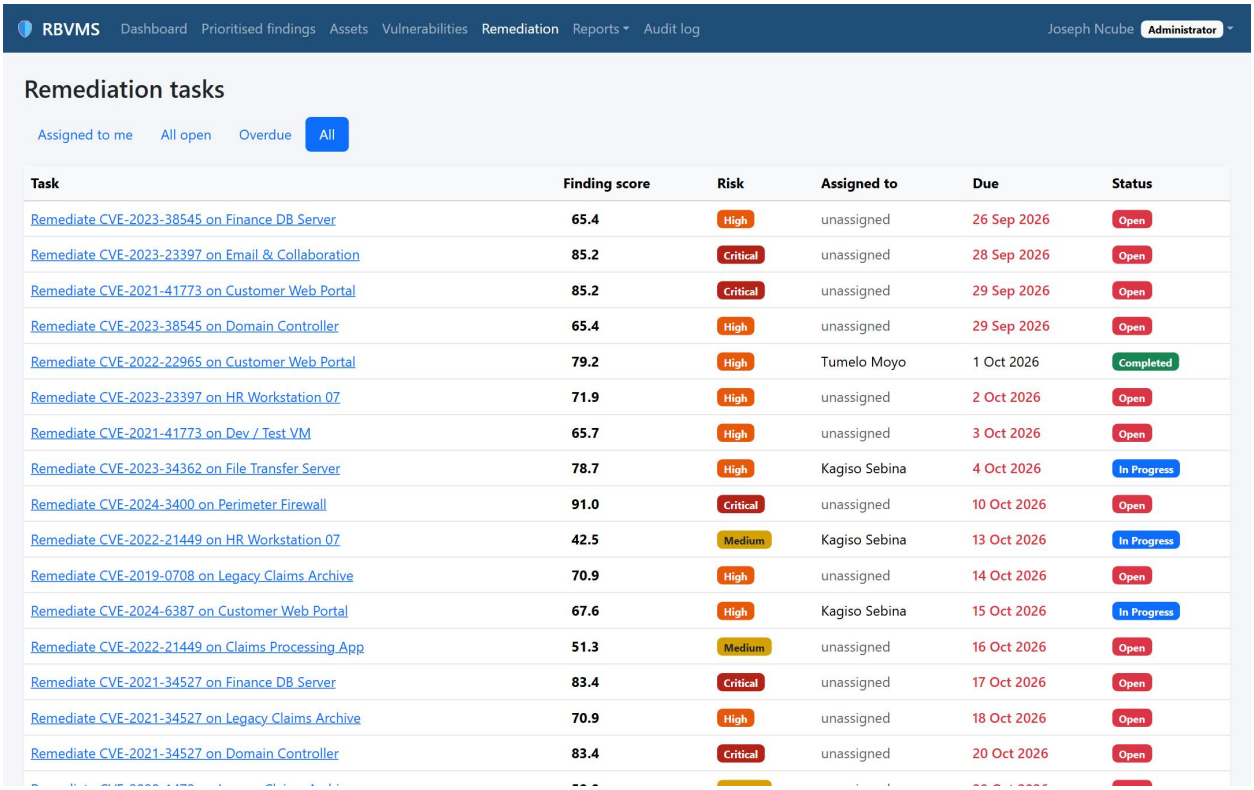


Fig 20. Remediation tasks with assignee, due date and overdue tasks highlighted.

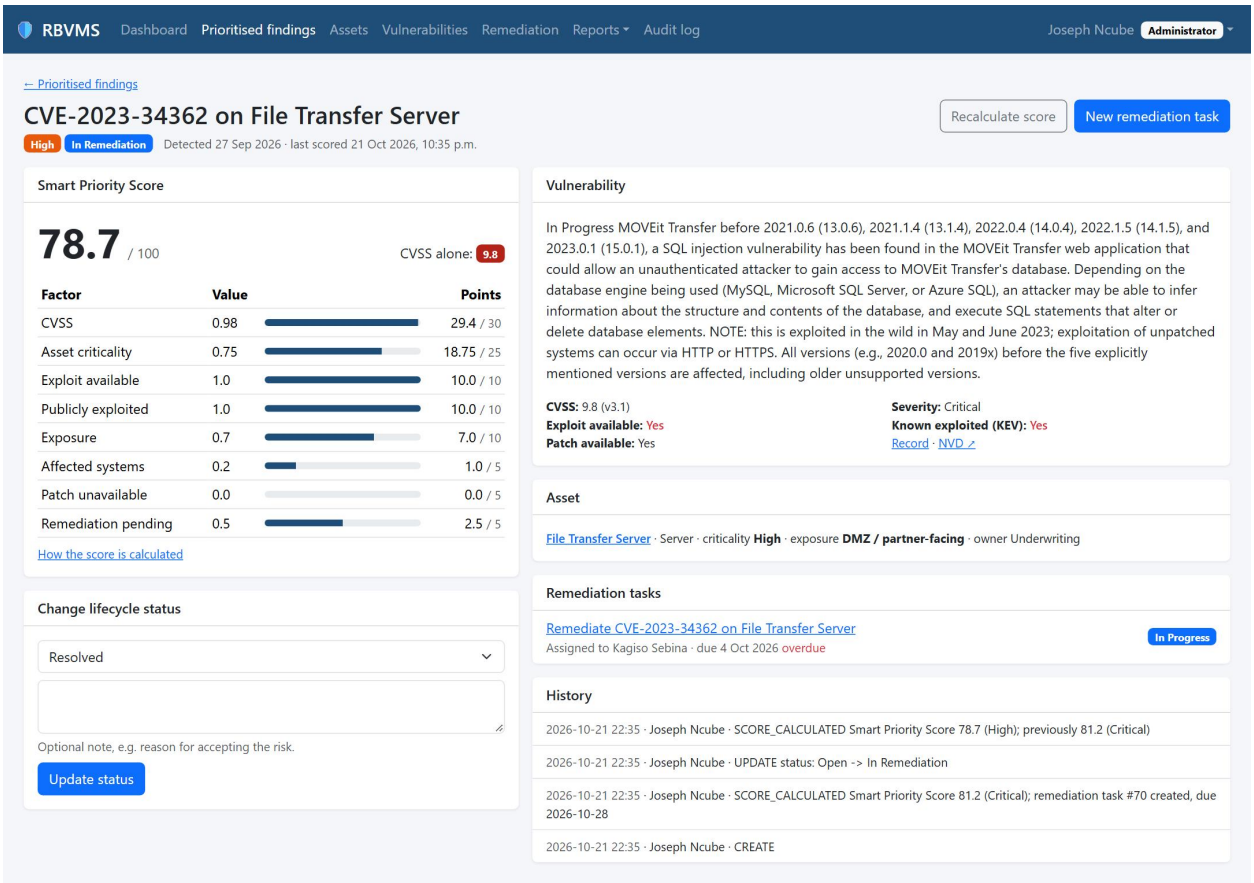


Fig 21. A finding in remediation (CVE-2023-34362 on the File Transfer Server) with its lifecycle status, assigned task and history.

4.1.7 Reporting, evaluation view and audit log

The reports page (Fig 22) summarizes findings by risk level and lifecycle status, active risk per asset, and remediation workload and completion per assignee; findings can also be exported as CSV. To support the evaluation planned in Section 2.2, an evaluation view (Fig 23) ranks the same active findings by Smart Priority Score and by CVSS alone and shows how far each finding moves. Every change to assets, vulnerabilities, findings and tasks, every sign-in and every score calculation is written to the audit log (Fig 24) with the user and time. Access is role-based: Administrators can manage all data, while Security Analysts can view everything but only update remediation tasks assigned to them.

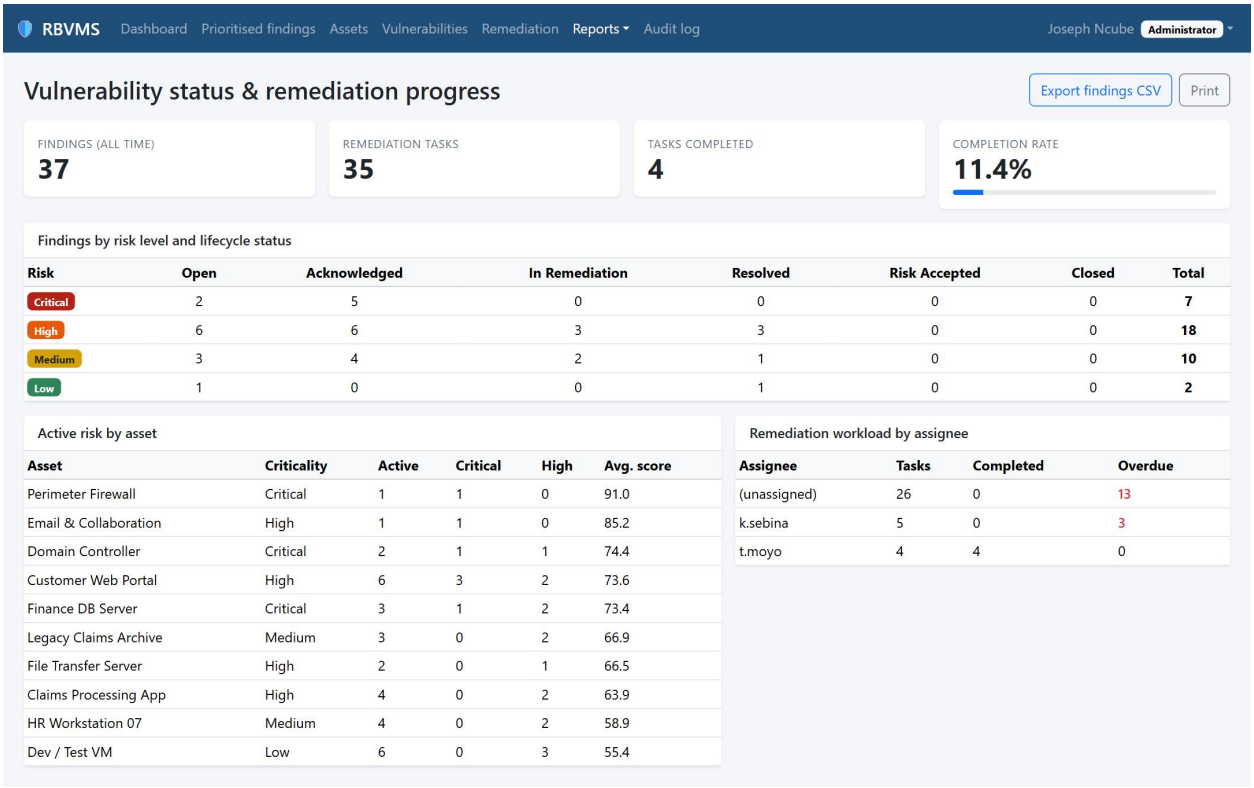


Fig 22. Vulnerability status and remediation progress report.

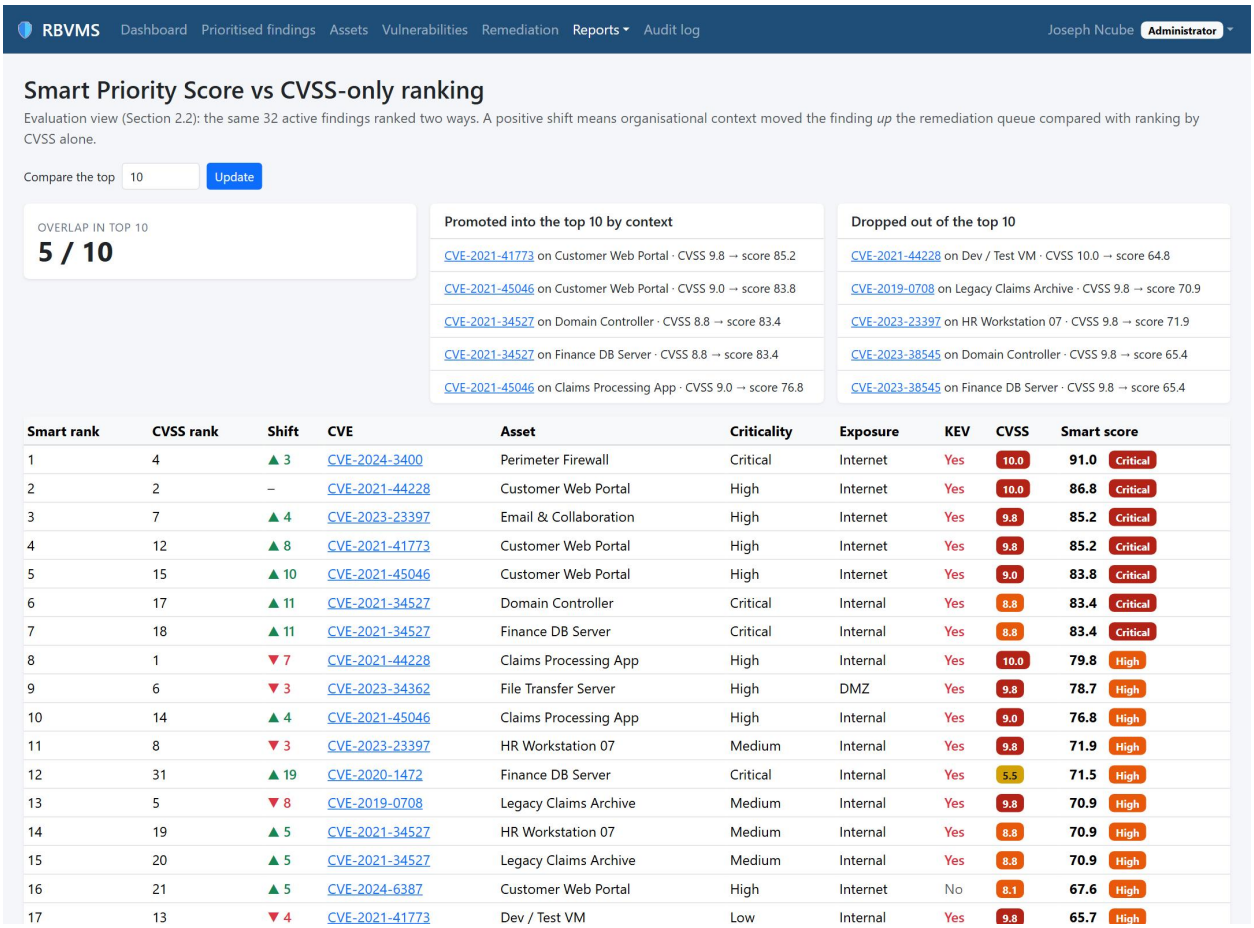


Fig 23. Evaluation view: the same active findings ranked by Smart Priority Score and by CVSS alone.

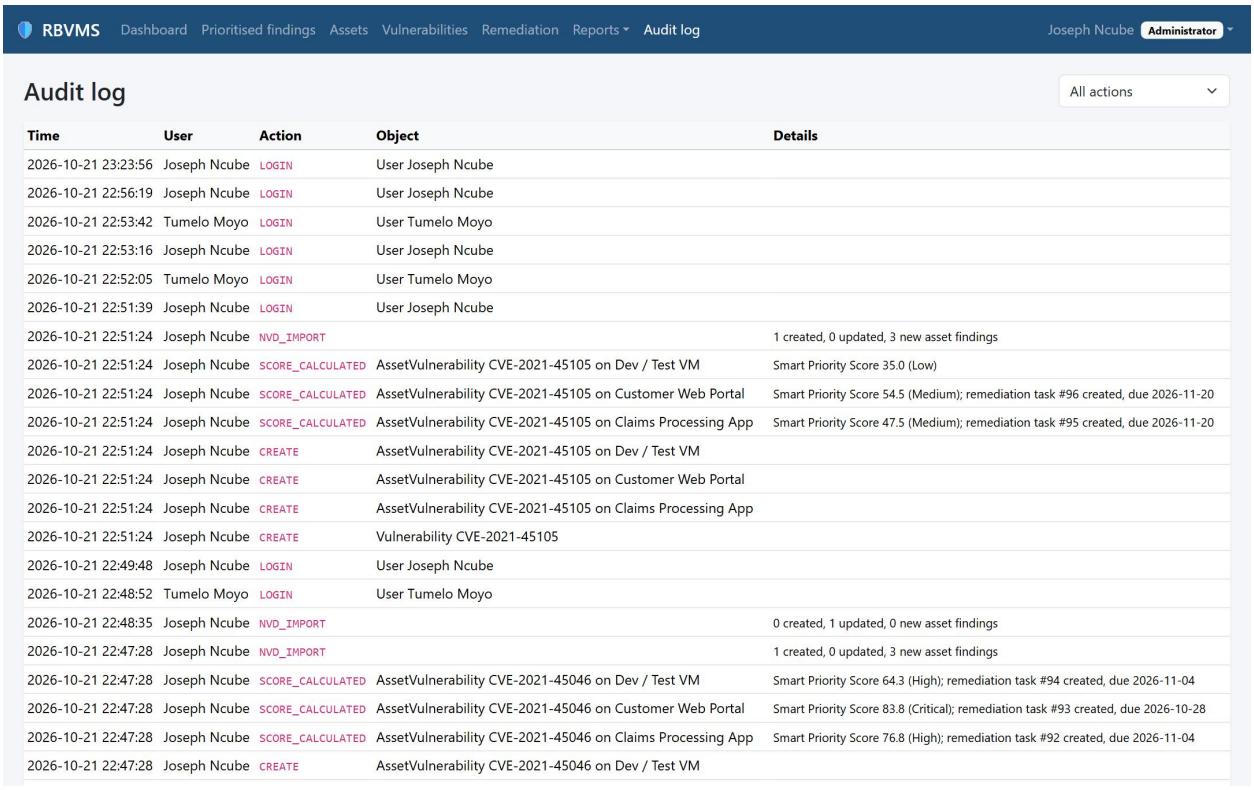


Fig 24. Audit log recording user actions with time and details.

4.1.8 Differences from the design

Most of the design in Chapters 1 to 3 was carried into the working system. Table 2 lists the elements that were changed, extended or not implemented.

Table 2. Differences between the Chapter 1 to 3 design and the implemented system.

Design element	Planned	As implemented
User class (Fig 5)	userId, username, role, passwordHash; login(), logout()	Extends Django's built-in user model with a role field. Login and logout use Django's authentication. A second role, Security Analyst, was added alongside Administrator.
Asset class (Fig 5)	name, criticality, assetType; addAsset(), updateCriticality()	As designed, plus exposure, owner, IP address and software keywords (used to match CVEs). Operations are web forms.
Vulnerability class (Fig 5)	cveId, cvssScore, description, exploitAvailable; importFromNVD()	As designed, plus CVSS vector, severity, KEV status, patch availability and affected products. Import is a separate service function rather than a method of the class.
calculateScore() (Section 3.7)	Creates the AssetVulnerability; creates a task when Medium or above	The AssetVulnerability is created when a CVE is matched to an asset, and calculateScore() then updates it. A task is only created if the finding has no open task, which avoids duplicates.
Sequence and communication diagrams (Figs 7, 8)	av1 sends messages to Vulnerability and Asset	The same steps in the same order, implemented as attribute access through Django's ORM.
State chart (Fig 9)	End state after Resolved or Risk Accepted	Implemented with an explicit Closed status.
Prioritization signals (Sections 1.10, 2.1)	CVE, CVSS, EPSS and CISA KEV studied	CVE, CVSS and KEV are used. **EPSS was not implemented.**
FR2 import schedule (Section 2.4)	Import on a scheduled or on-demand basis	On-demand import (web page and import_nvd command). **No schedule has been configured.**
Testing tools (Section 1.11)	Django test framework / pytest-django and Postman	pytest-django used. **Postman was not used**, as the system offers no REST API of its own.
Testing (Section 1.10)	Unit, integration and user acceptance testing	Unit and integration tests and black-box system tests. **No user acceptance testing with real administrators.**

4.2 Testing techniques

The system was tested with black-box testing: inputs were entered through the web interface exactly as a user would enter them, and the outputs shown on screen were checked against the expected behaviour, without reference to the internal code. The expected result of every scoring test was calculated by hand from the published weights in Table 1 before the test was run. To make the tests repeatable, the browser steps were scripted with Playwright, which drives a real Chromium browser, types into the forms, clicks

the buttons and reads the result from the page. A screenshot was taken for every test case. Test data was fabricated: the assets named "TC ..." and the vulnerabilities CVE-TEST-0001 and CVE-TEST-0002 exist only for testing. Table 3 summarizes the results; each test case is then shown in turn.

Table 3. Black-box test cases and results.

ID	Test	Expected result	Actual result	Result
TC-01	Sign in with a wrong password	Sign-in refused, "Invalid username or password."	Message shown; user stayed on the sign-in page	Pass
TC-02	Analyst opens an Administrator page	HTTP 403 "Access denied"; no import form	HTTP 403, "Access denied", no form	Pass
TC-03	NVD import with no criteria	Form rejected with a validation message	Validation message shown	Pass
TC-04	Import CVE-2021-44832 from the NVD	CVSS equals the NVD website (6.6); matched to the 3 Log4j assets	CVSS 6.6, same vector; 1 record, 3 scored findings	Pass
TC-05	Score CVE-TEST-0001 on a Critical, Internet-facing asset	93.0 Critical; Open task due in 7 days	93.0 Critical; Open task due 28 Oct 2026 (detected 21 Oct)	Pass
TC-06	Add a second affected asset (Medium, DMZ)	New asset 78.5 High; first asset re-scored to 94.0	78.5 High and 94.0 Critical	Pass
TC-07	Lower the first asset to Medium, Internal	74.5 High, below the second asset; drops out of the dashboard top 10	Rank 1 before, not in top 10 after; 74.5 below 78.5	Pass
TC-08	Acknowledge and assign the task	Status In Remediation; 78.5 becomes 76.0	In Remediation, 76.0, task with Tumelo Moyo	Pass
TC-09	Score a low-severity, patched CVE	30.5 Low; no remediation task	30.5 Low; no task	Pass
TC-10	Accept the risk and close the finding	Closed; score 25.5; no further status changes offered	Closed, 25.5, no status form	Pass
TC-11	Audit log after TC-07	Entry showing the criticality and exposure changes	Both changes logged, but also an unchanged IP address field (defect D-01)	Pass with defect
TC-11R	Retest after fixing D-01	Only the real change is logged	Only "criticality: Medium -> High"	Pass
TC-12	NFR 1: 5,000 records in under 5 s	Every load under 5 s	6.48, 7.60 and 8.80 s (defect D-02)	Fail
TC-12R	Retest after the D-02 fix	Every load under 5 s	4.21, 3.80 and 5.81 s	Fail

4.2.1 Access control and input validation

TC-01 entered the administrator's username with a wrong password; the system refused the sign-in (Fig 25). TC-02 signed in as the Security Analyst t.moyo and opened the Administrator-only import page; the system returned HTTP 403 and an "Access denied" page instead of the form (Fig 26). TC-03 submitted the NVD import form with every search field empty; the form was rejected with a validation message and nothing was imported (Fig 27). All three passed.

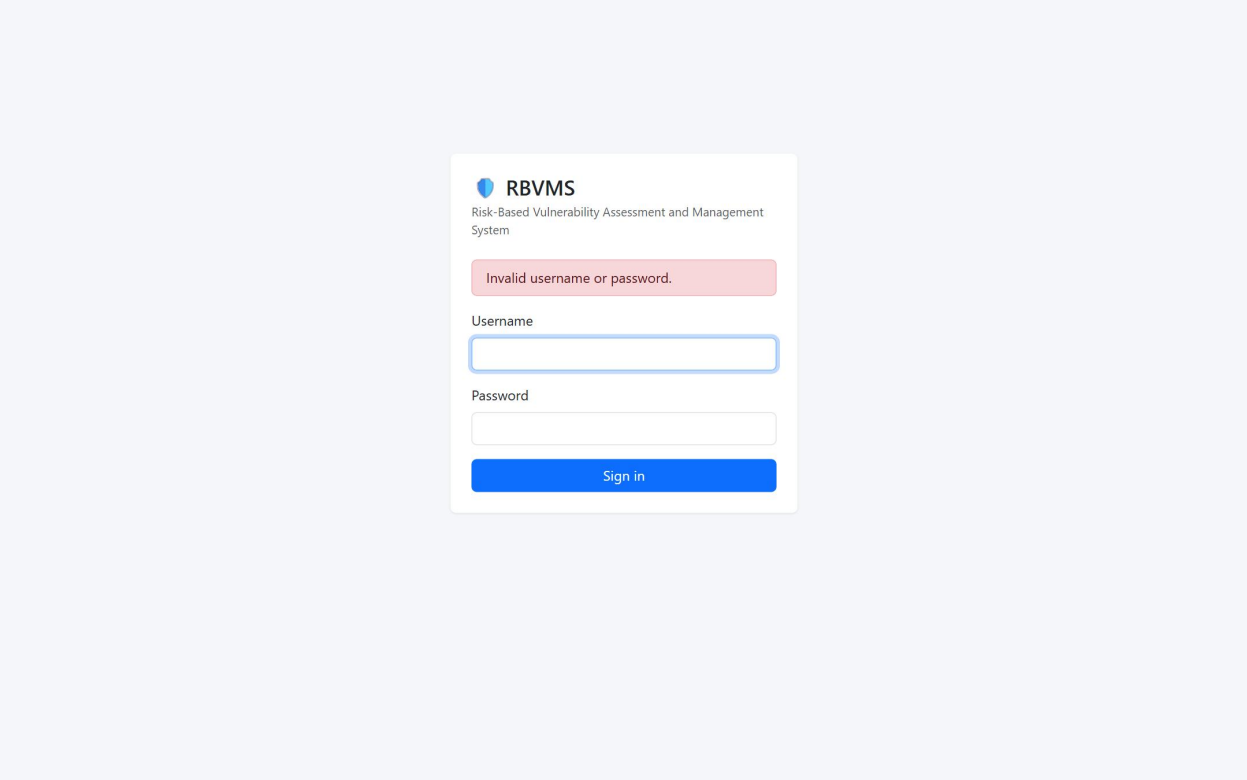


Fig 25. TC-01: sign-in with a wrong password is refused.

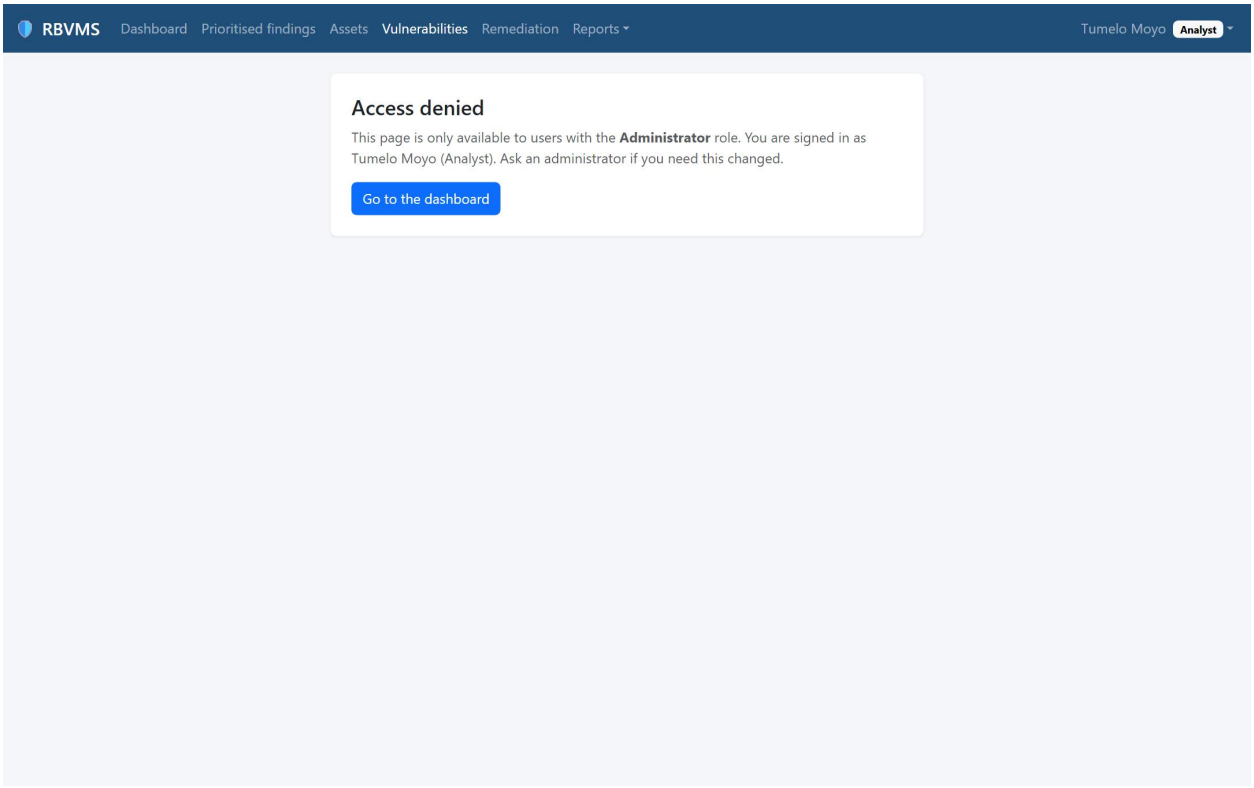


Fig 26. TC-02: a Security Analyst is refused the Administrator-only import page.

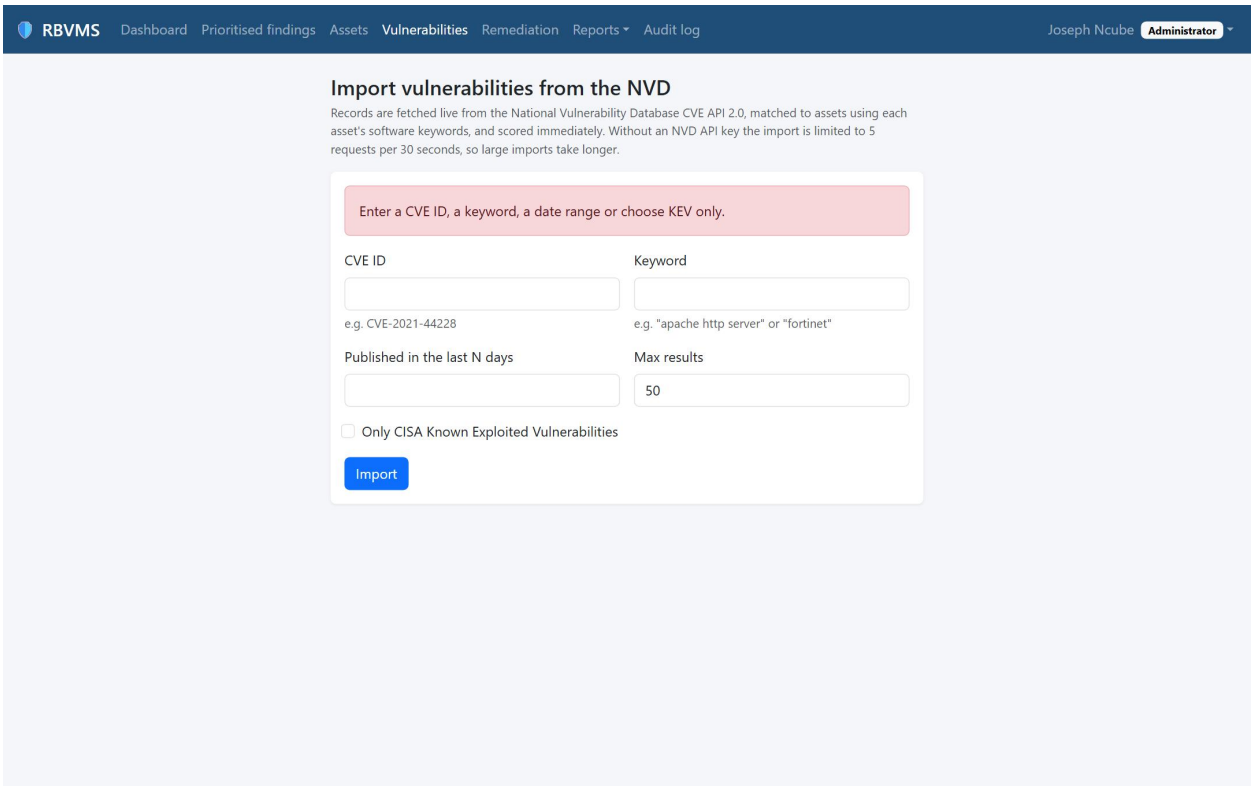


Fig 27. TC-03: an NVD import with no search criteria is rejected.

4.2.2 Vulnerability ingestion

TC-04 checked that imported data is accurate by comparing it with an independent source, the NVD website. The NVD publishes a CVSS v3.1 base score of 6.6 (Medium) for CVE-2021-44832 (Fig 28). The CVE ID was entered in the import form; the system reported one new record and three new scored findings (Fig 29), and the imported record shows the same score of 6.6 and the same vector string, and is matched to the three assets whose software includes Log4j (Fig 30). The test passed.

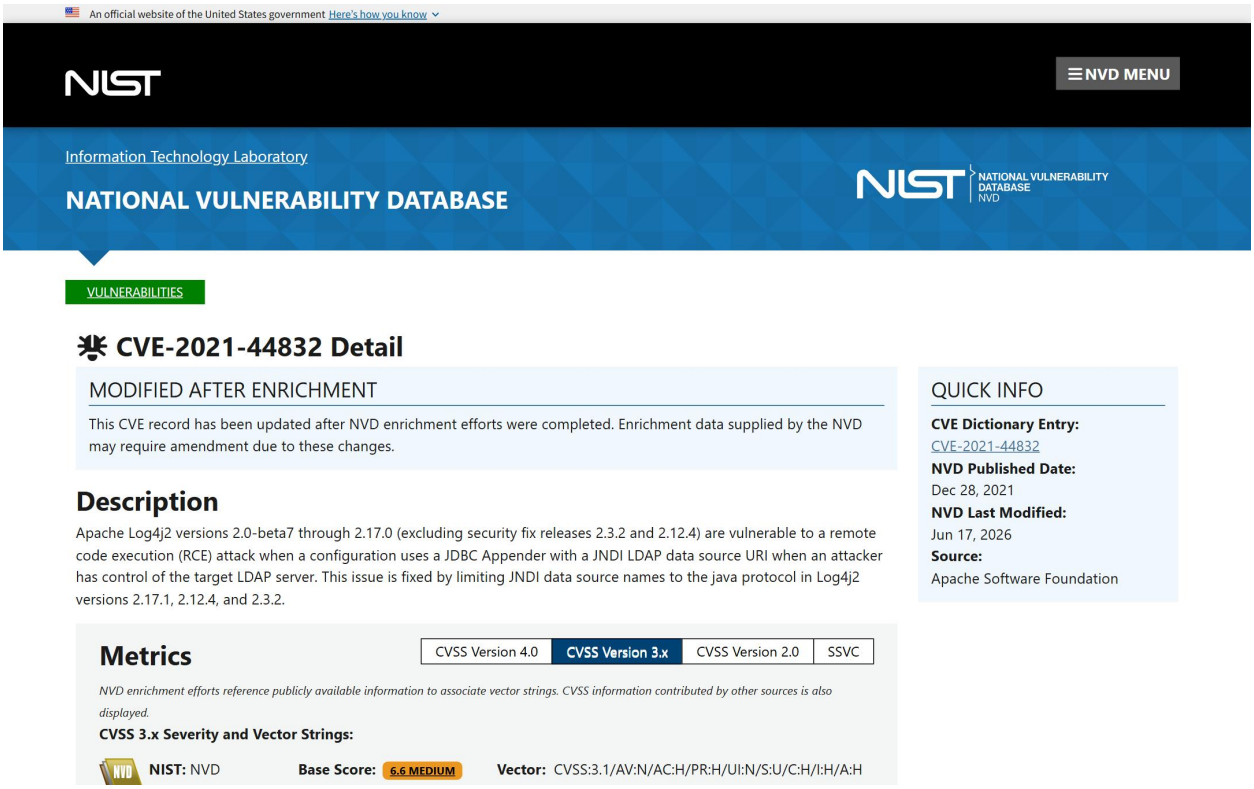


Fig 28. TC-04 expected value: the NVD website lists CVE-2021-44832 with a CVSS v3.1 base score of 6.6.

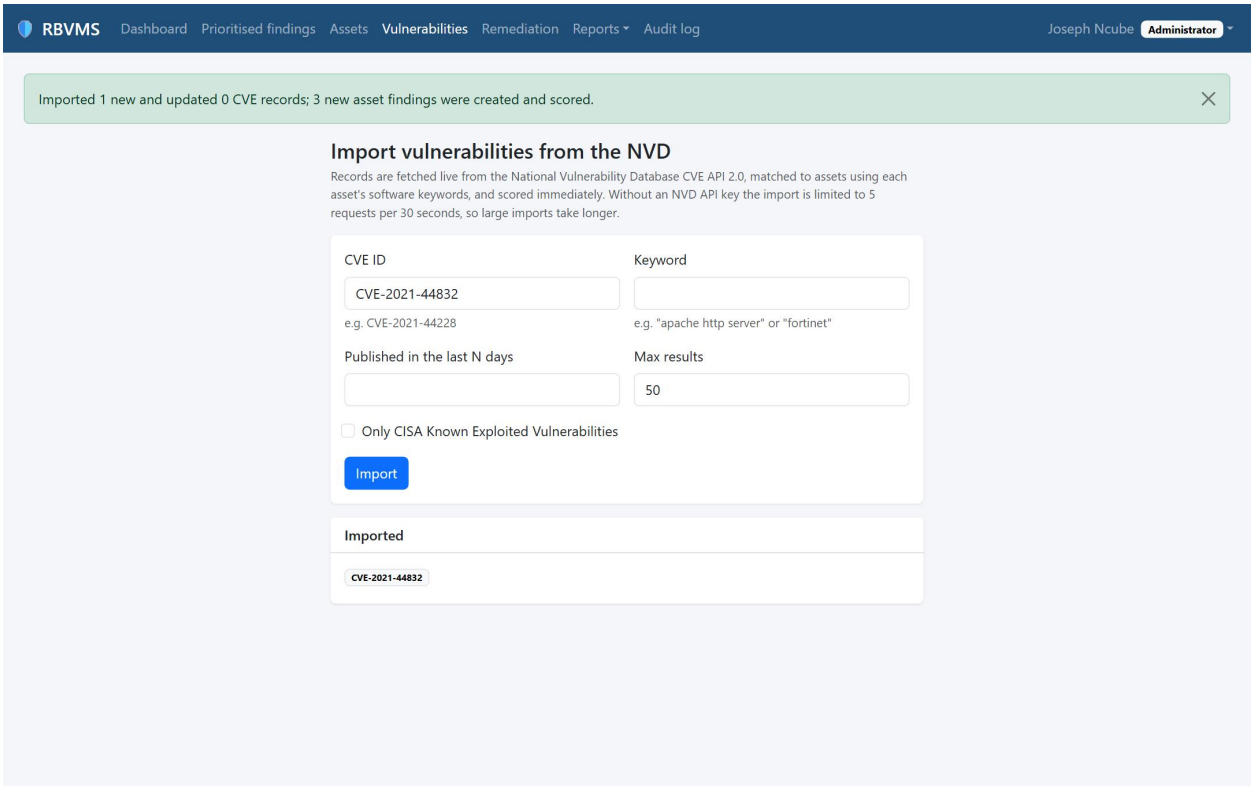


Fig 29. TC-04 input and output: CVE-2021-44832 imported through the web form.

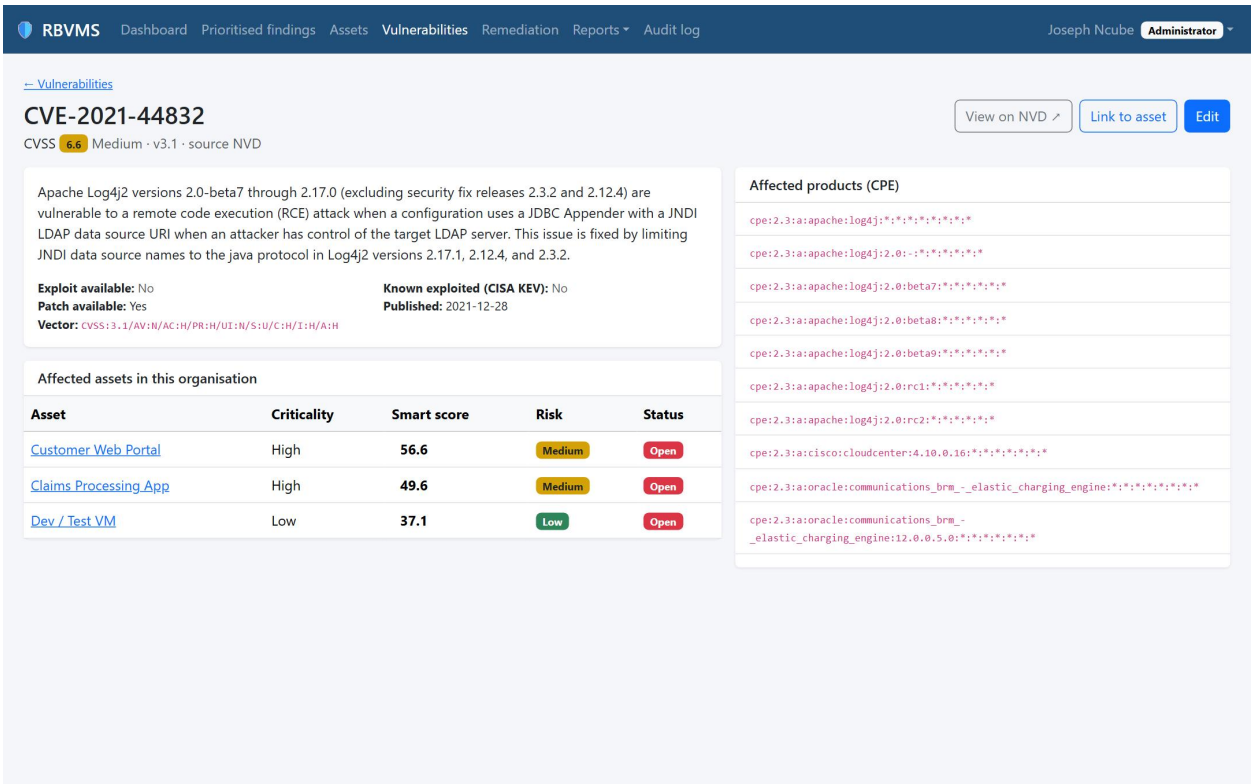


Fig 30. TC-04 output: the imported record shows CVSS 6.6 with the same vector and three affected assets.

4.2.3 Smart Priority Score calculation

TC-05 tested the score against known inputs. An asset was entered with criticality Critical and exposure Internet-facing (Fig 31), and a vulnerability with CVSS 9.0, a public exploit, KEV listing and no patch (Fig 32). From Table 1 the expected score is 27.0 + 25 + 10 + 10 + 10 + 1.0 + 5 + 5 = 93.0, which is Critical, with an Open task due seven days after detection. The system showed exactly 93.0, Critical, and an Open task due on 28 October 2026 for a finding detected on 21 October 2026 (Fig 33).

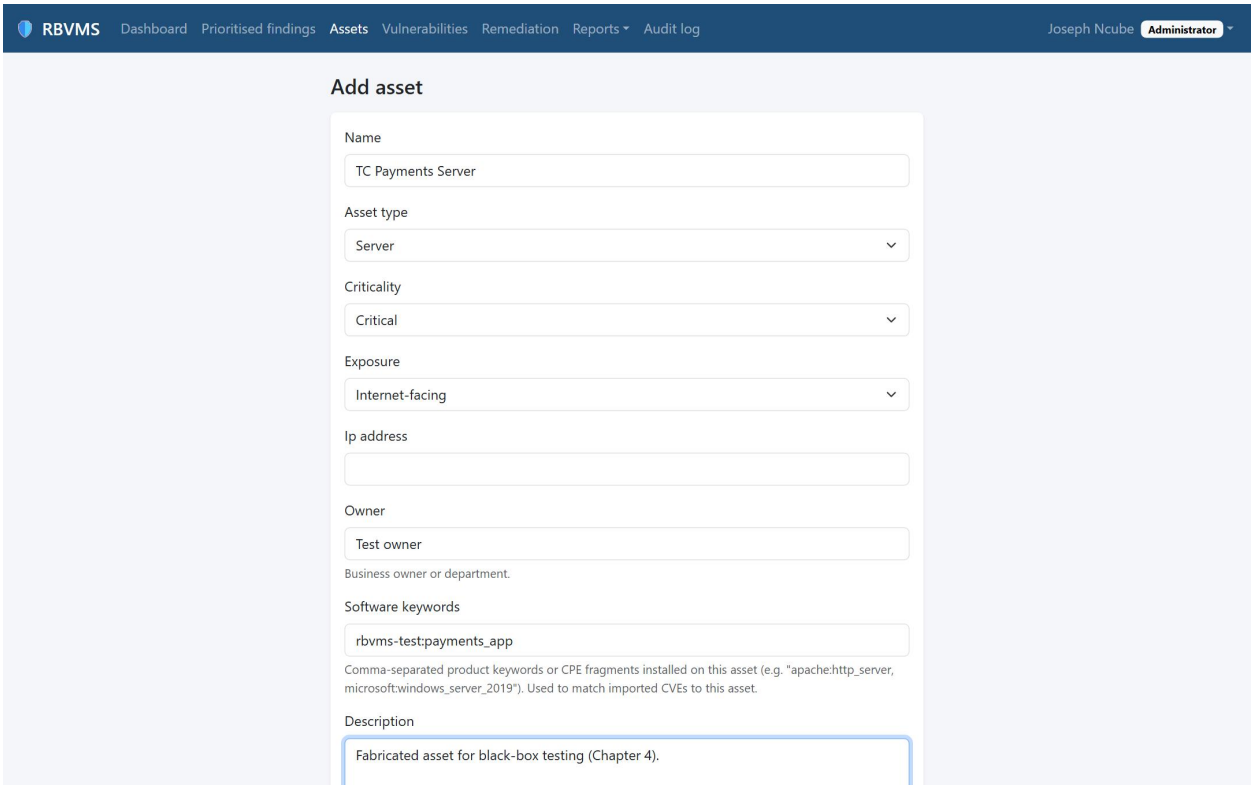


Fig 31. TC-05 input: asset TC Payments Server, criticality Critical, exposure Internet-facing.

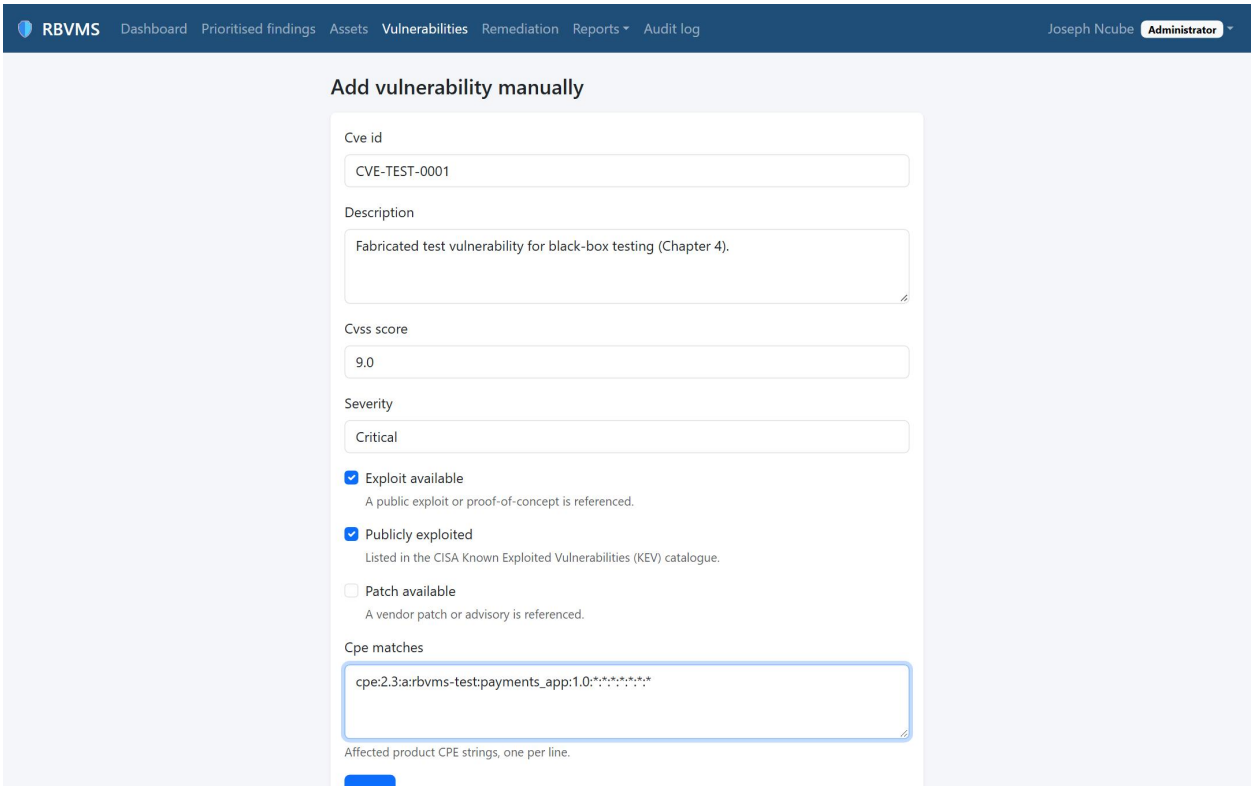


Fig 32. TC-05 input: vulnerability CVE-TEST-0001, CVSS 9.0, exploit available, in KEV, no patch.

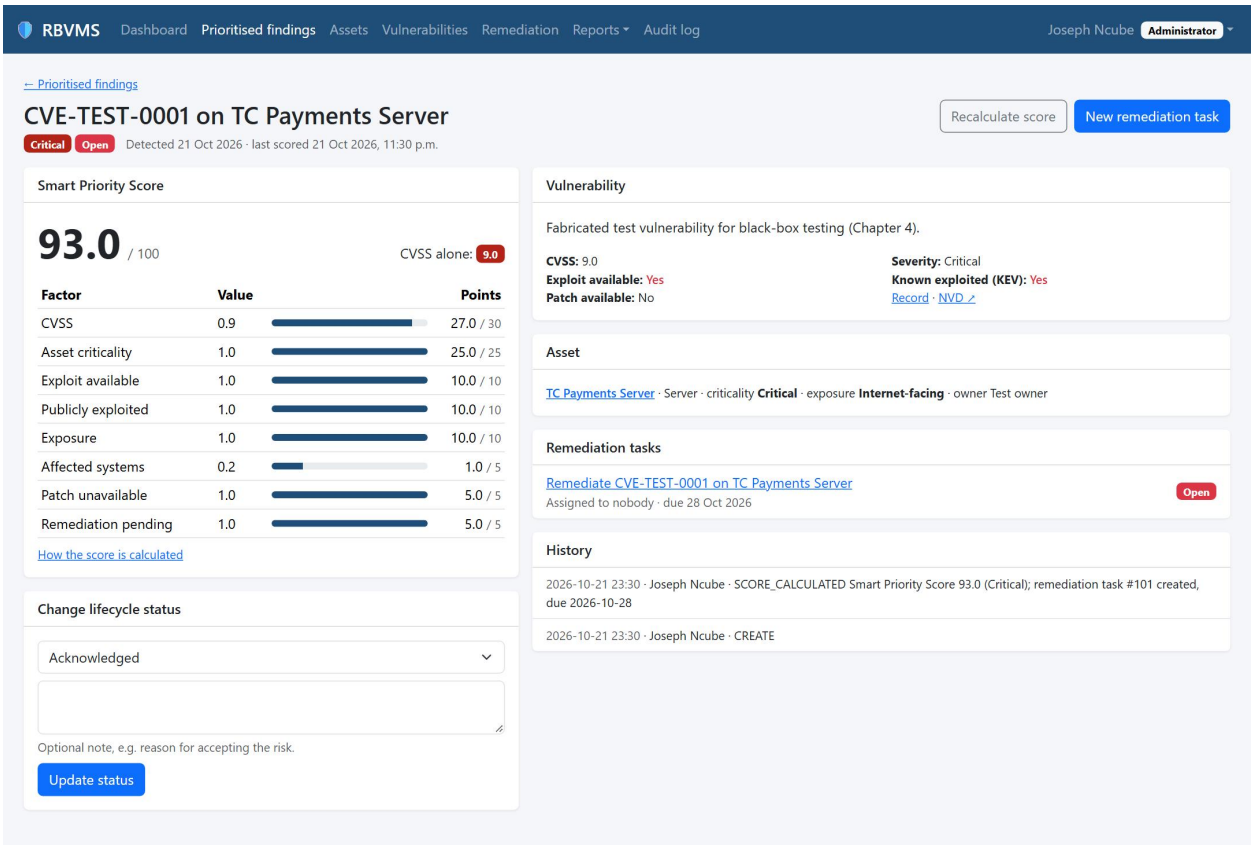


Fig 33. TC-05 output: Smart Priority Score 93.0 (Critical) with an Open task due in seven days.

TC-06 added a second asset with the same software, criticality Medium and exposure DMZ. Its expected score is 27.0 + 12.5 + 10 + 10 + 7.0 + 2.0 + 5 + 5 = 78.5 (High). Because two assets are now affected, the first asset's score should also rise by one point to 94.0. The prioritized list showed both values, in that order (Fig 34).

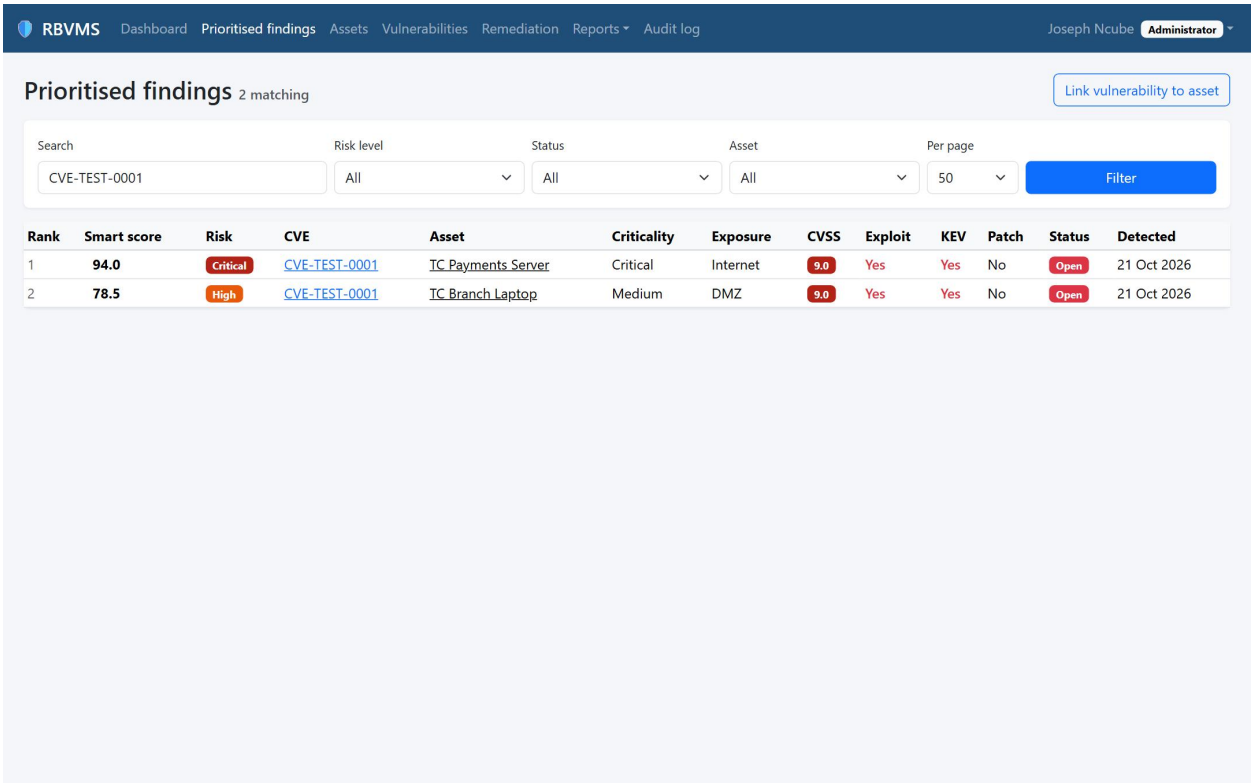


Fig 34. TC-06 output: the second asset scores 78.5 (High) and the first is re-scored to 94.0.

4.2.4 Dashboard re-ranking when inputs change

TC-07 tested whether the dashboard re-prioritizes when an organizational input changes. Before the change, the TC Payments Server finding was ranked first on the dashboard with 94.0 (Fig 35). The asset was then edited from Critical and Internet-facing to Medium and Internal (Fig 36). The expected new score is 27.0 + 12.5 + 10 + 10 + 3.0 + 2.0 + 5 + 5 = 74.5, below the second asset's 78.5 and below the tenth dashboard entry. After saving, the finding no longer appeared in the dashboard's top ten, where the second asset now held tenth place (Fig 37), and the prioritized list showed 74.5 ranked below 78.5 (Fig 38). The test passed.

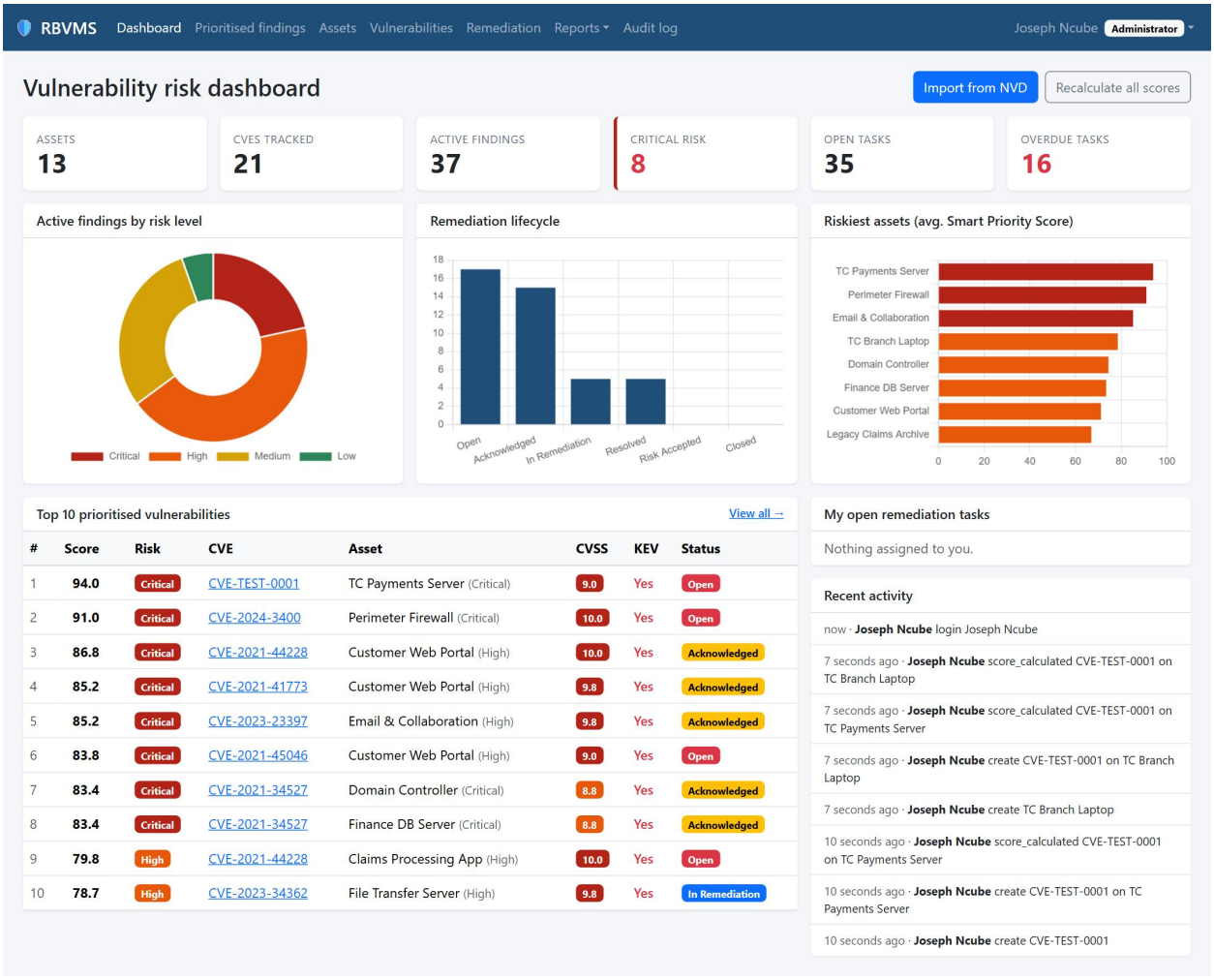


Fig 35. TC-07 before: TC Payments Server ranked first on the dashboard with 94.0.

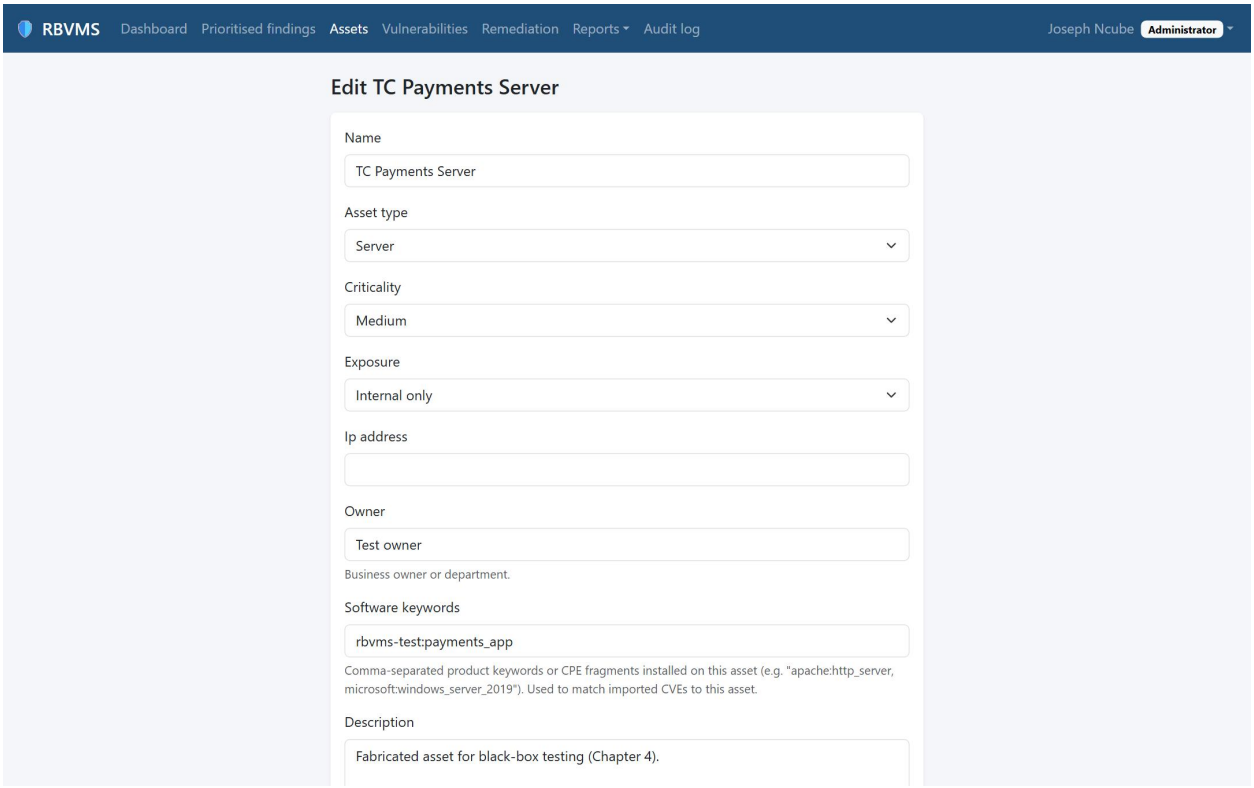


Fig 36. TC-07 input: asset changed to criticality Medium and exposure Internal.

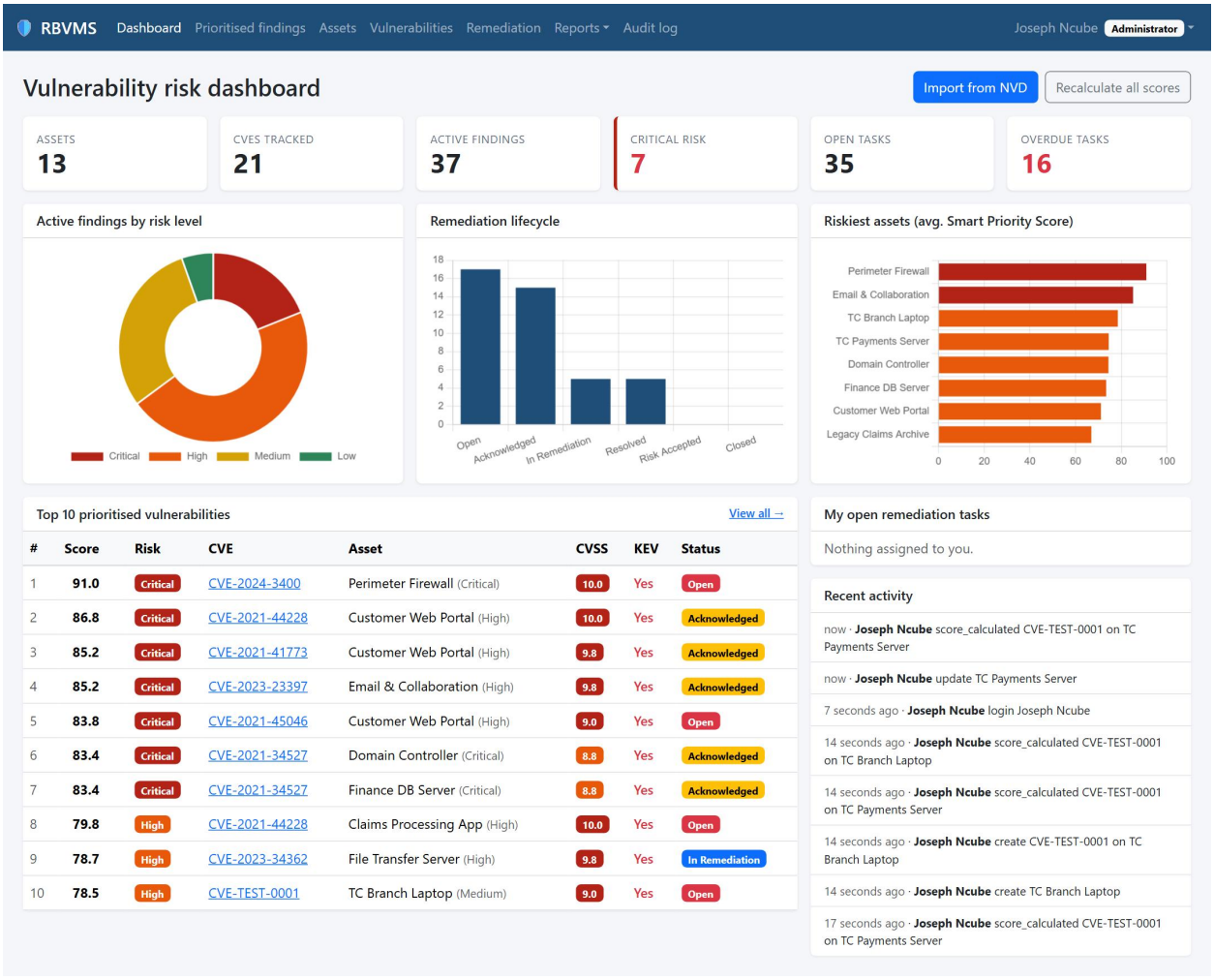


Fig 37. TC-07 after: TC Payments Server has dropped out of the top ten; TC Branch Laptop is tenth.

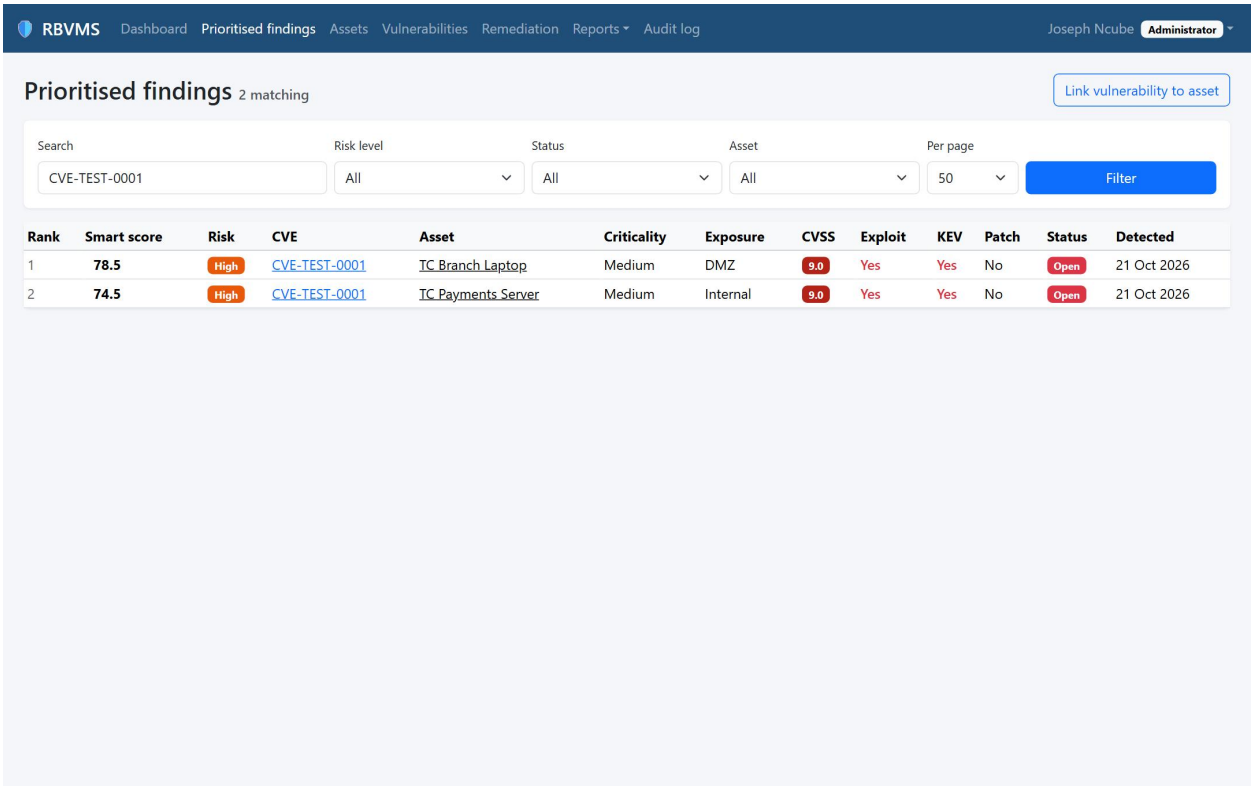


Fig 38. TC-07 after: the prioritized list shows TC Payments Server at 74.5, below TC Branch Laptop at 78.5.

4.2.5 Remediation workflow

TC-08 acknowledged the TC Branch Laptop finding and assigned its task to Tumelo Moyo with status In Progress (Fig 39). According to the state chart the finding should move to In Remediation, and the remediation-pending factor should halve from 5 to 2.5, giving 76.0. The system showed In Remediation and 76.0 (Fig 40).

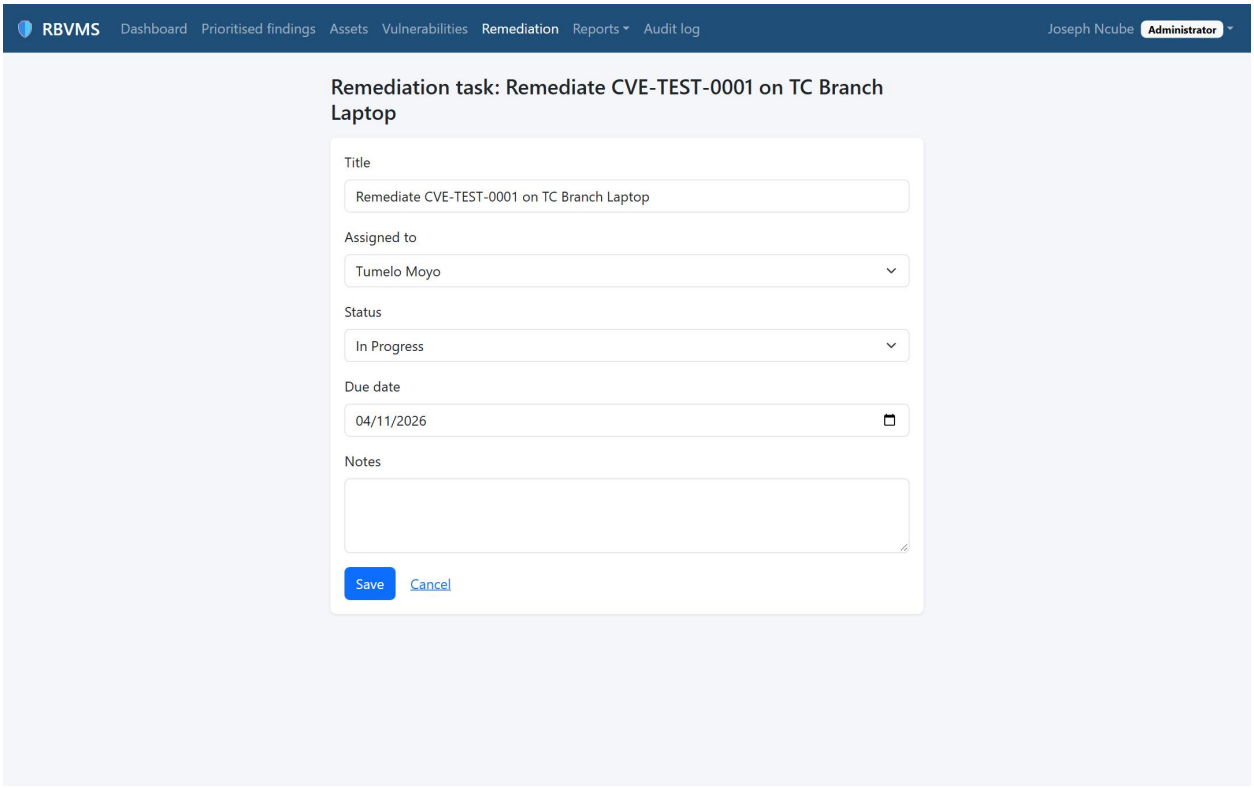


Fig 39. TC-08 input: remediation task assigned to Tumelo Moyo, status In Progress.

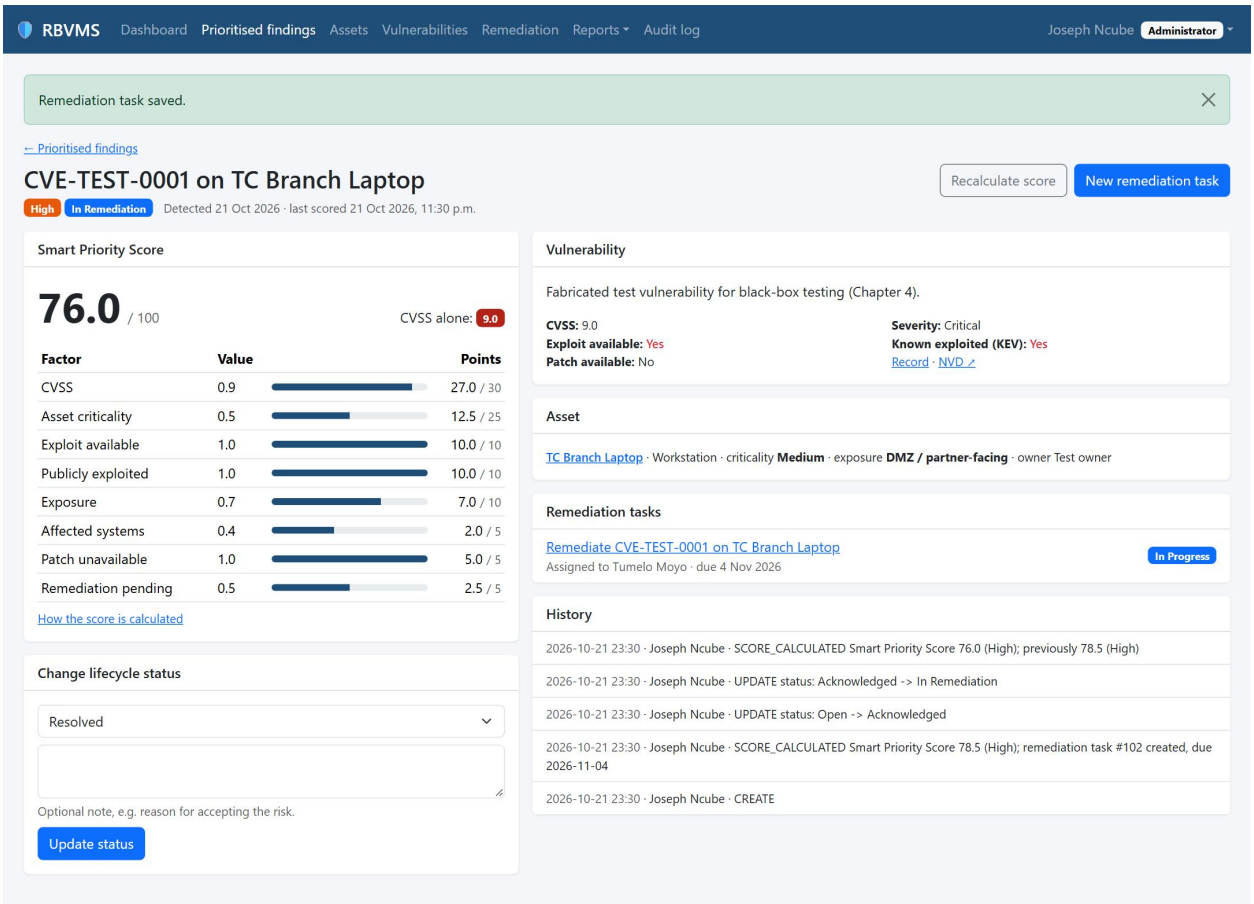


Fig 40. TC-08 output: finding In Remediation with its score lowered from 78.5 to 76.0.

TC-09 entered a low-severity, patched vulnerability (CVSS 3.0, no exploit, not in KEV) on a Medium, internal asset (Fig 41). The expected score is 9.0 + 12.5 + 0 + 0 + 3.0 + 1.0 + 0 + 5 = 30.5, which is Low, so no remediation task should be created. The system showed 30.5 (Low) and no task (Fig 42). TC-10 then took the same finding along the risk-acceptance path of the state chart: Acknowledged, Risk Accepted with a note, then Closed. The score fell to the expected 25.5, and no further status changes were offered once the finding was closed (Fig 43).

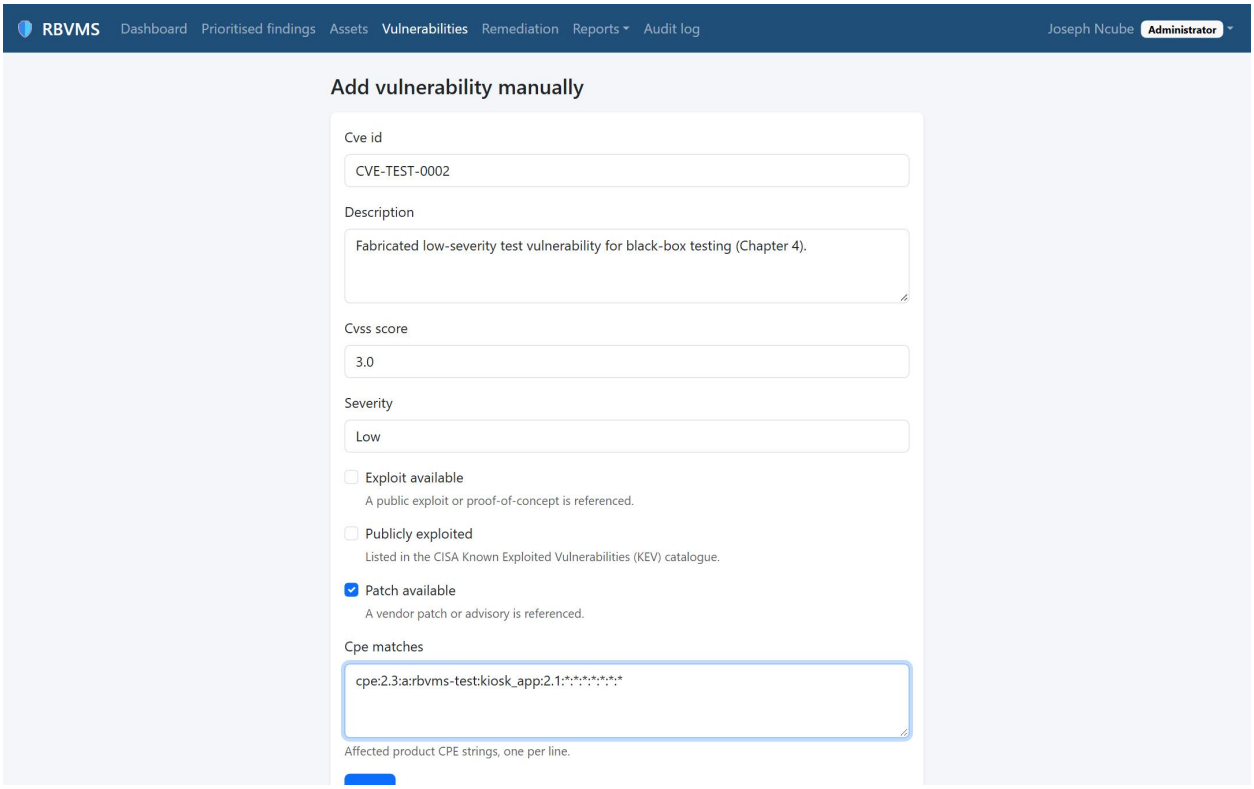


Fig 41. TC-09 input: CVE-TEST-0002, CVSS 3.0, no exploit, not in KEV, patch available.

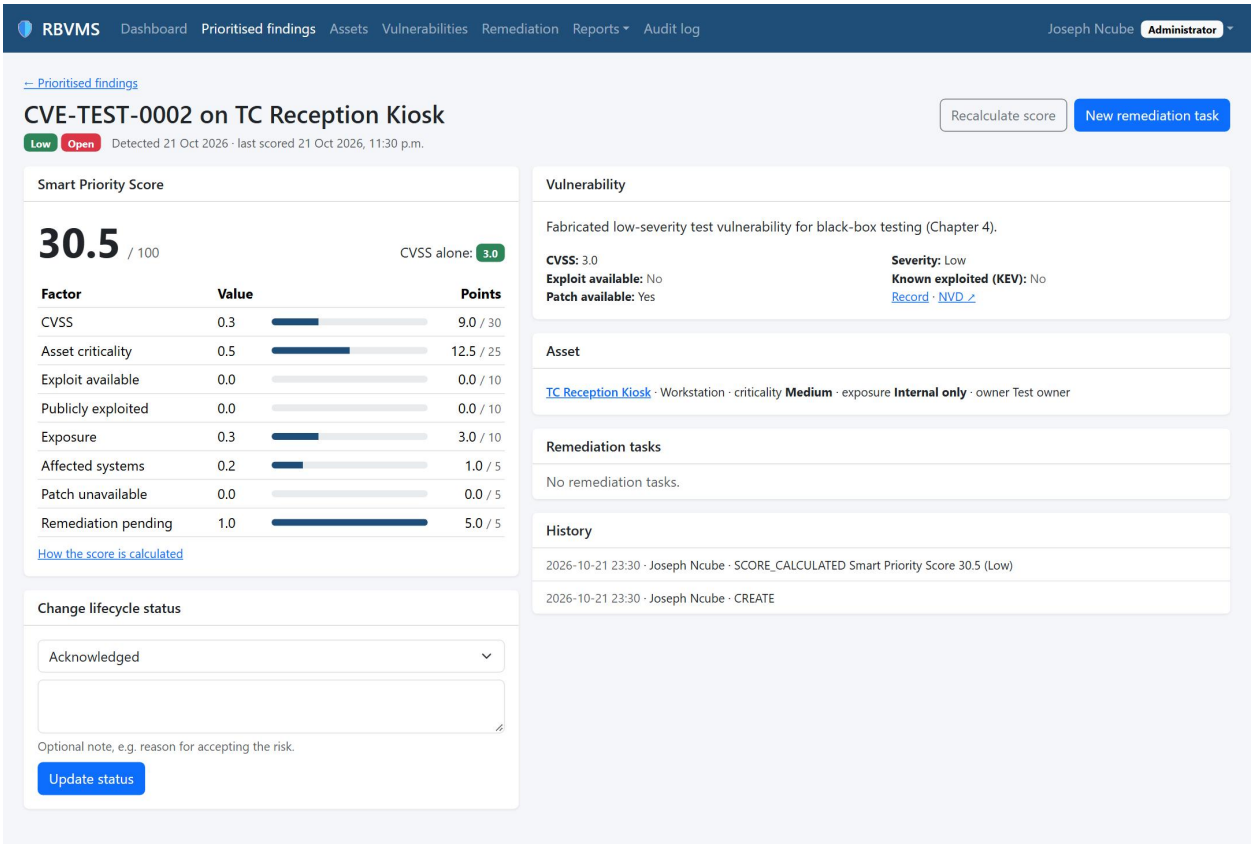


Fig 42. TC-09 output: Smart Priority Score 30.5 (Low) and no remediation task.

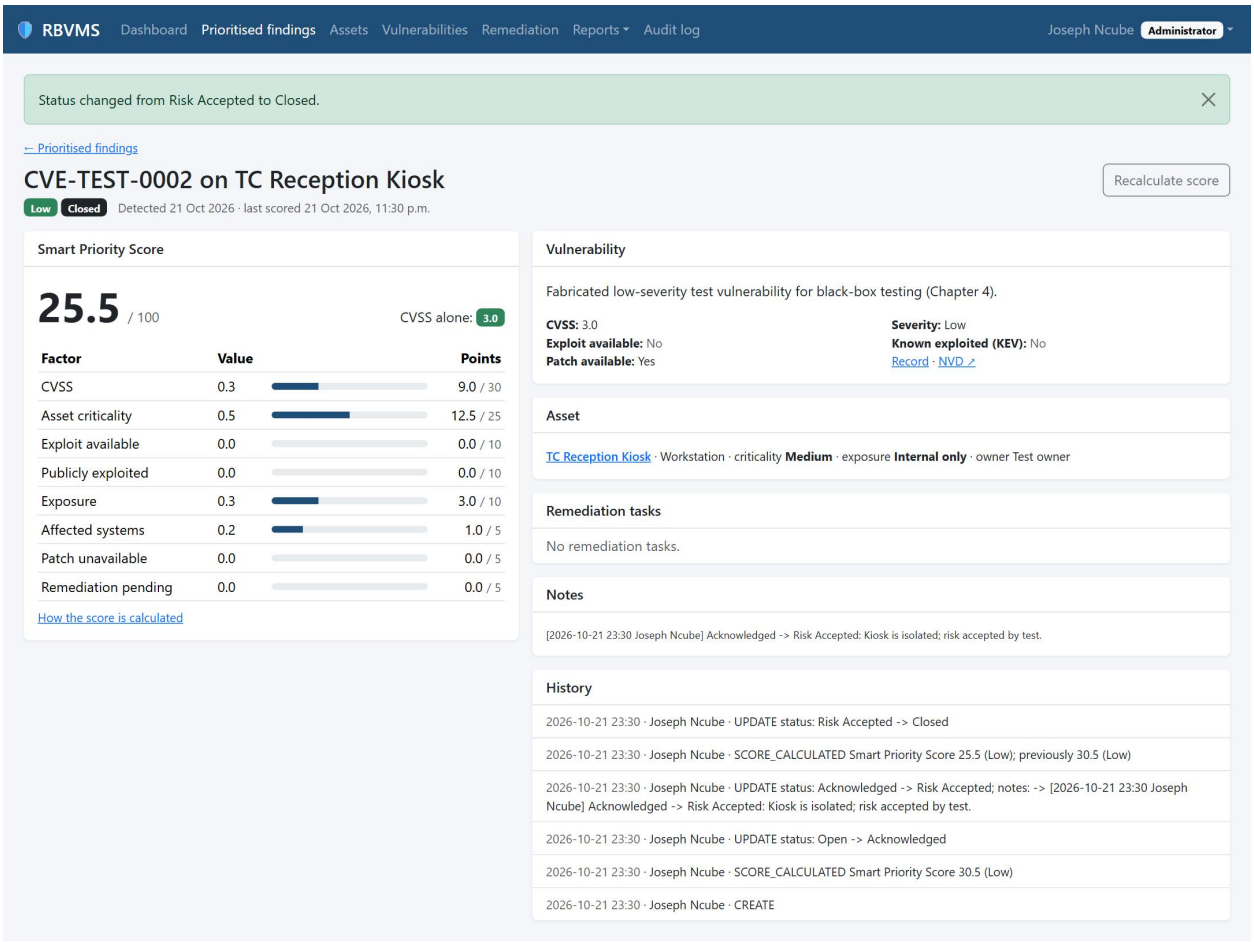
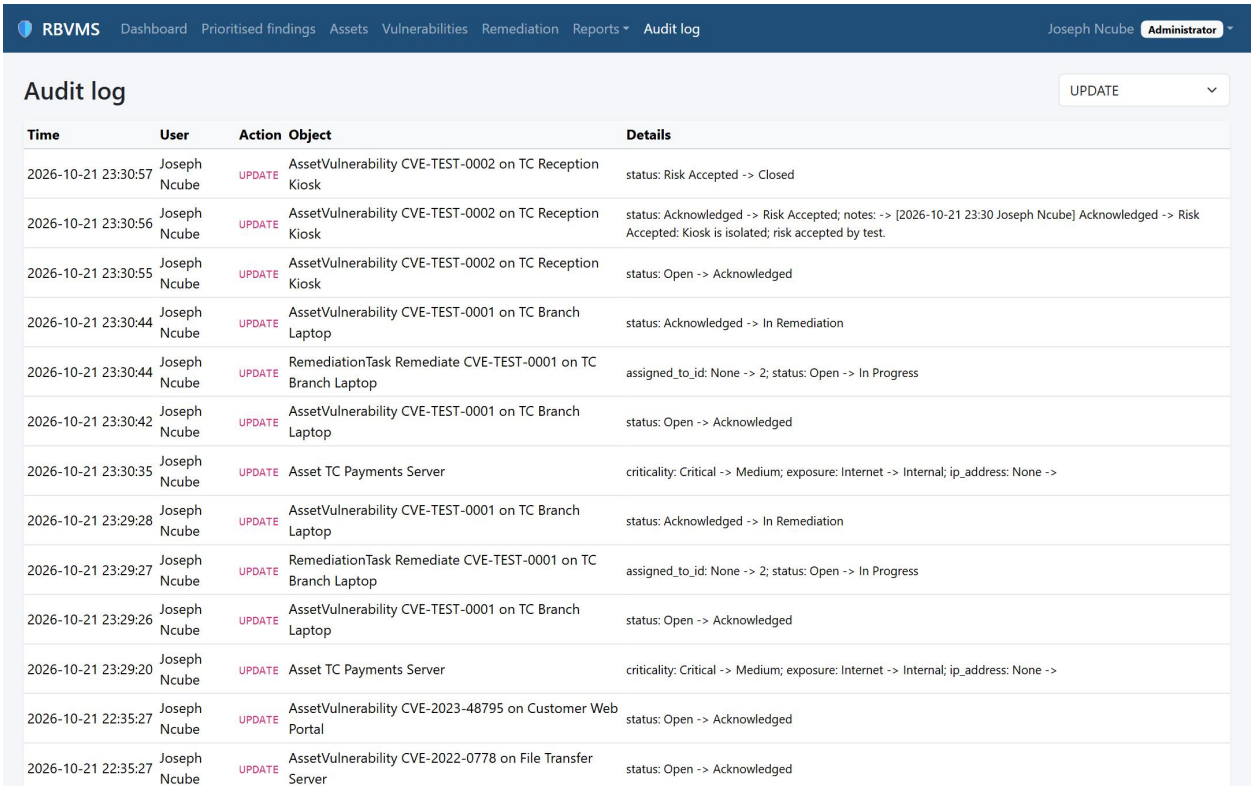


Fig 43. TC-10 output: the finding accepted and closed, with its score at 25.5 and the full history.

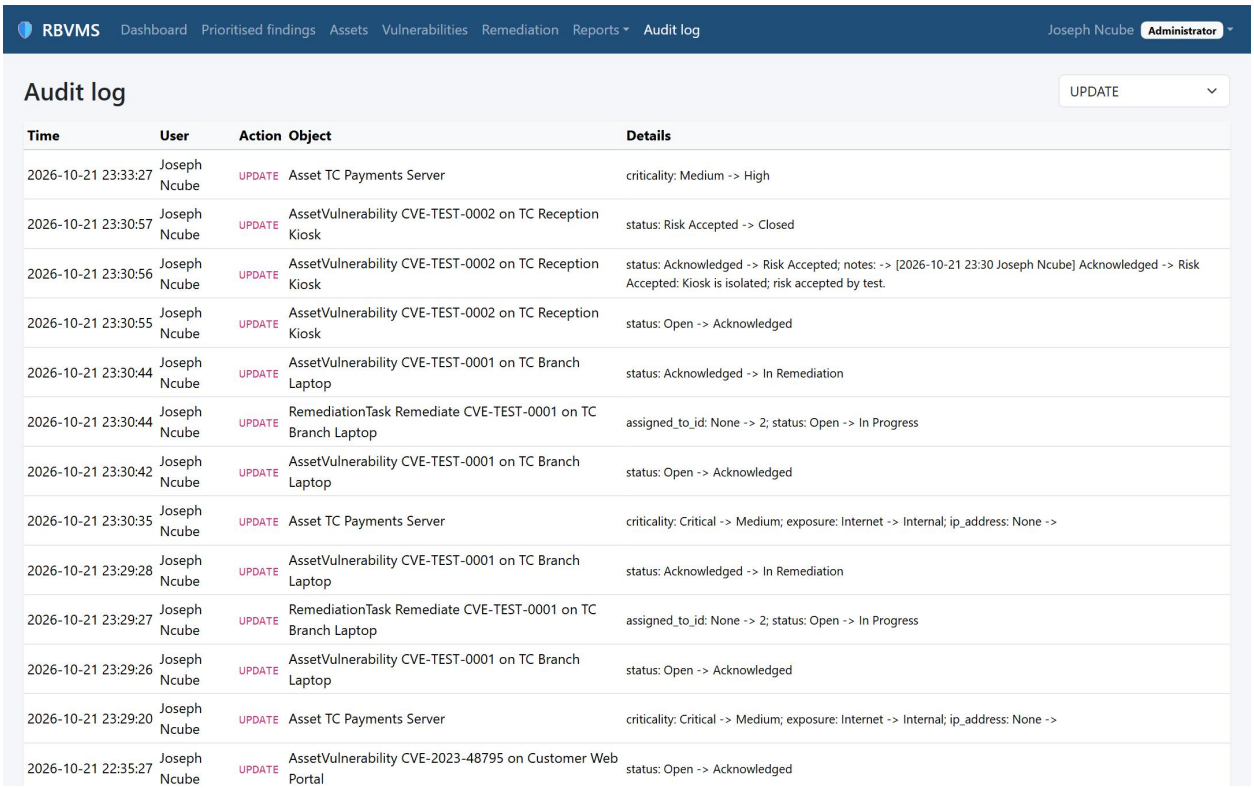
4.2.6 Audit log and defect D-01

TC-11 checked the audit entry for the TC-07 change. The entry recorded the user, the time and both real changes (criticality Critical to Medium, exposure Internet to Internal), but it also listed the IP address field as changed from "None" to blank, although the IP address had not been touched (Fig 44). This was recorded as defect D-01: an empty optional field submitted by a form was compared with the stored empty value and reported as a change, adding noise to the audit trail. The comparison was corrected to treat the two as equal, and a regression check was added to the automated tests. In retest TC-11R the asset's criticality was changed from Medium to High, and the new entry listed only that change (Fig 45).



Time	User	Action	Object	Details
2026-10-21 23:30:57	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0002 on TC Reception Kiosk	status: Risk Accepted -> Closed
2026-10-21 23:30:56	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0002 on TC Reception Kiosk	status: Acknowledged -> Risk Accepted; notes: -> [2026-10-21 23:30 Joseph Ncube] Acknowledged -> Risk Accepted: Kiosk is isolated; risk accepted by test.
2026-10-21 23:30:55	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0002 on TC Reception Kiosk	status: Open -> Acknowledged
2026-10-21 23:30:44	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0001 on TC Branch Laptop	status: Acknowledged -> In Remediation
2026-10-21 23:30:44	Joseph Ncube	UPDATE	RemediationTask Remediate CVE-TEST-0001 on TC Branch Laptop	assigned_to_id: None -> 2; status: Open -> In Progress
2026-10-21 23:30:42	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0001 on TC Branch Laptop	status: Open -> Acknowledged
2026-10-21 23:30:35	Joseph Ncube	UPDATE	Asset TC Payments Server	criticality: Critical -> Medium; exposure: Internet -> Internal; ip_address: None ->
2026-10-21 23:29:28	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0001 on TC Branch Laptop	status: Acknowledged -> In Remediation
2026-10-21 23:29:27	Joseph Ncube	UPDATE	RemediationTask Remediate CVE-TEST-0001 on TC Branch Laptop	assigned_to_id: None -> 2; status: Open -> In Progress
2026-10-21 23:29:26	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0001 on TC Branch Laptop	status: Open -> Acknowledged
2026-10-21 23:29:20	Joseph Ncube	UPDATE	Asset TC Payments Server	criticality: Critical -> Medium; exposure: Internet -> Internal; ip_address: None ->
2026-10-21 22:35:27	Joseph Ncube	UPDATE	AssetVulnerability CVE-2023-48795 on Customer Web Portal	status: Open -> Acknowledged
2026-10-21 22:35:27	Joseph Ncube	UPDATE	AssetVulnerability CVE-2022-0778 on File Transfer Server	status: Open -> Acknowledged

Fig 44. TC-11 output: the audit entry includes the real changes and a spurious IP address change (defect D-01).



Time	User	Action	Object	Details
2026-10-21 23:33:27	Joseph Ncube	UPDATE	Asset TC Payments Server	criticality: Medium -> High
2026-10-21 23:30:57	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0002 on TC Reception Kiosk	status: Risk Accepted -> Closed
2026-10-21 23:30:56	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0002 on TC Reception Kiosk	status: Acknowledged -> Risk Accepted; notes: -> [2026-10-21 23:30 Joseph Ncube] Acknowledged -> Risk Accepted: Kiosk is isolated; risk accepted by test.
2026-10-21 23:30:55	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0002 on TC Reception Kiosk	status: Open -> Acknowledged
2026-10-21 23:30:44	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0001 on TC Branch Laptop	status: Acknowledged -> In Remediation
2026-10-21 23:30:44	Joseph Ncube	UPDATE	RemediationTask Remediate CVE-TEST-0001 on TC Branch Laptop	assigned_to_id: None -> 2; status: Open -> In Progress
2026-10-21 23:30:42	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0001 on TC Branch Laptop	status: Open -> Acknowledged
2026-10-21 23:30:35	Joseph Ncube	UPDATE	Asset TC Payments Server	criticality: Critical -> Medium; exposure: Internet -> Internal; ip_address: None ->
2026-10-21 23:29:28	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0001 on TC Branch Laptop	status: Acknowledged -> In Remediation
2026-10-21 23:29:27	Joseph Ncube	UPDATE	RemediationTask Remediate CVE-TEST-0001 on TC Branch Laptop	assigned_to_id: None -> 2; status: Open -> In Progress
2026-10-21 23:29:26	Joseph Ncube	UPDATE	AssetVulnerability CVE-TEST-0001 on TC Branch Laptop	status: Open -> Acknowledged
2026-10-21 23:29:20	Joseph Ncube	UPDATE	Asset TC Payments Server	criticality: Critical -> Medium; exposure: Internet -> Internal; ip_address: None ->
2026-10-21 22:35:27	Joseph Ncube	UPDATE	AssetVulnerability CVE-2023-48795 on Customer Web Portal	status: Open -> Acknowledged

Fig 45. TC-11R output: after the fix, the entry lists only the real change.

4.2.7 Performance and defect D-02

TC-12 tested non-functional requirement NFR 1 (Section 2.4.1): a prioritized list of 5,000 records should render in under 5 seconds on the development hardware. About 5,000 synthetic findings were loaded and the full list was opened three times in the browser, measuring the complete page load. The loads took 6.48, 7.60 and 8.80 seconds, so the test failed (Fig 46). This was recorded as defect D-02. Timing each phase showed that the server spent about 2.3 seconds building the page, the uncompressed page was 3.4 MB, and the browser needed about 3 seconds to parse and lay out 5,000 table rows. The rows are now built directly in Python rather than through the template engine, which halved the server time to about 1.1 seconds, and responses are compressed, which reduced the page to 82 KB. In retest TC-12R the loads took 4.21, 3.80 and 5.81 seconds (Fig 47). This is a clear improvement, but one load still exceeded 5 seconds, so NFR 1 is **not reliably met**. The remaining time is mostly the browser rendering 5,000 rows, measured on Django's development server; the further changes needed are discussed in Section 4.3. The figures carry a small timing label added by the test script.

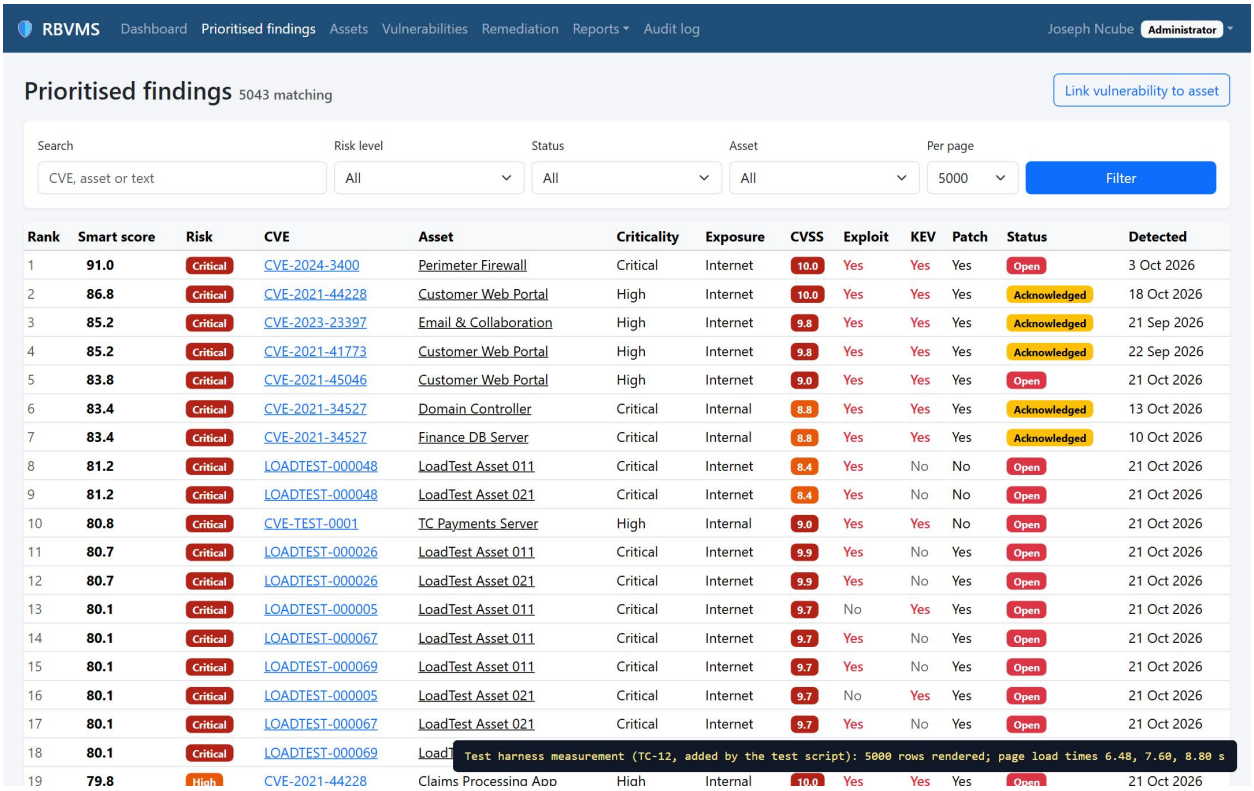


Fig 46. TC-12 output: 5,000 findings rendered; load times 6.48, 7.60 and 8.80 s (fail, defect D-02).

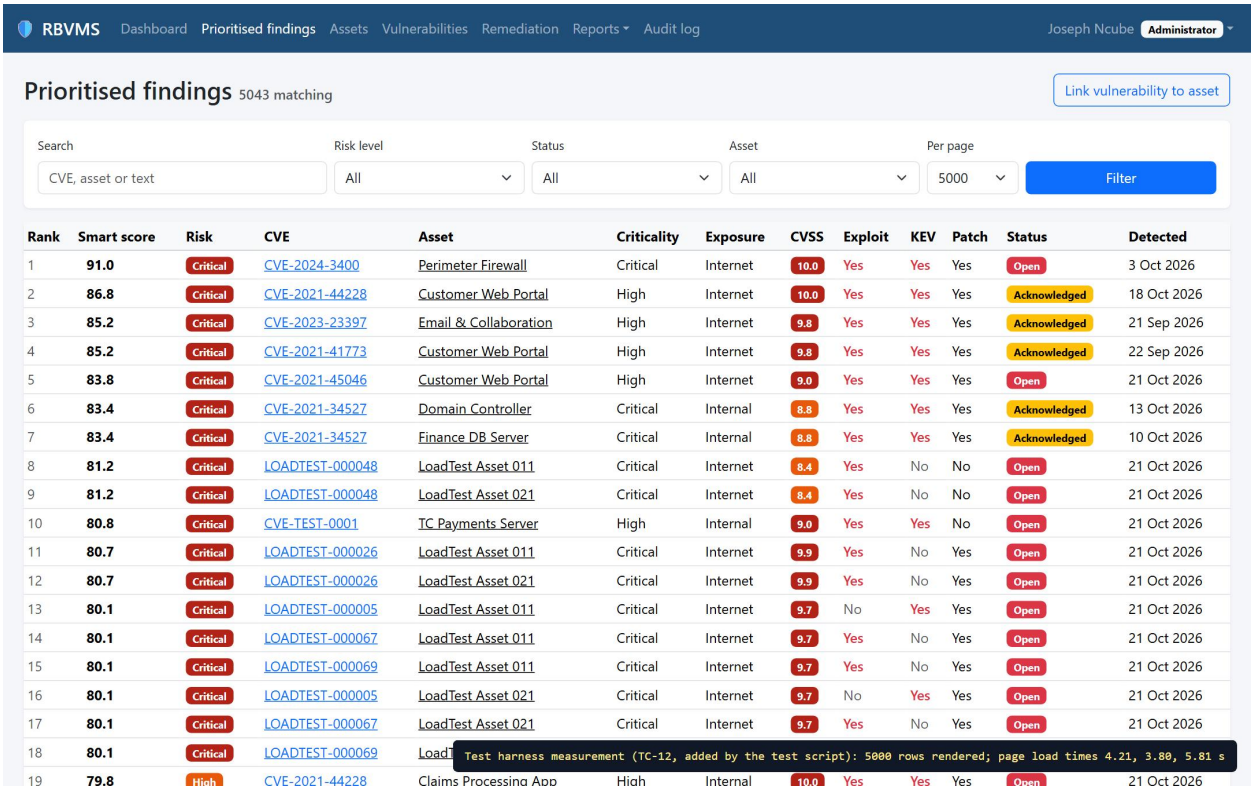


Fig 47. TC-12R output: after the fix, load times 4.21, 3.80 and 5.81 s (still a fail).

4.2.8 Supporting automated tests

Alongside the black-box tests, 61 automated unit and integration tests written with pytest-django were run against MySQL after all fixes, and all passed (Fig 48). They cover the score calculation and the pre- and post-conditions of Section 3.7, repeatability of the score (NFR 3, test_score_is_repeatable), salted password hashing (NFR 2, test_passwords_are_hashed), NVD parsing and retries, the state chart and role-based access. Table 4 lists the defects found during testing.

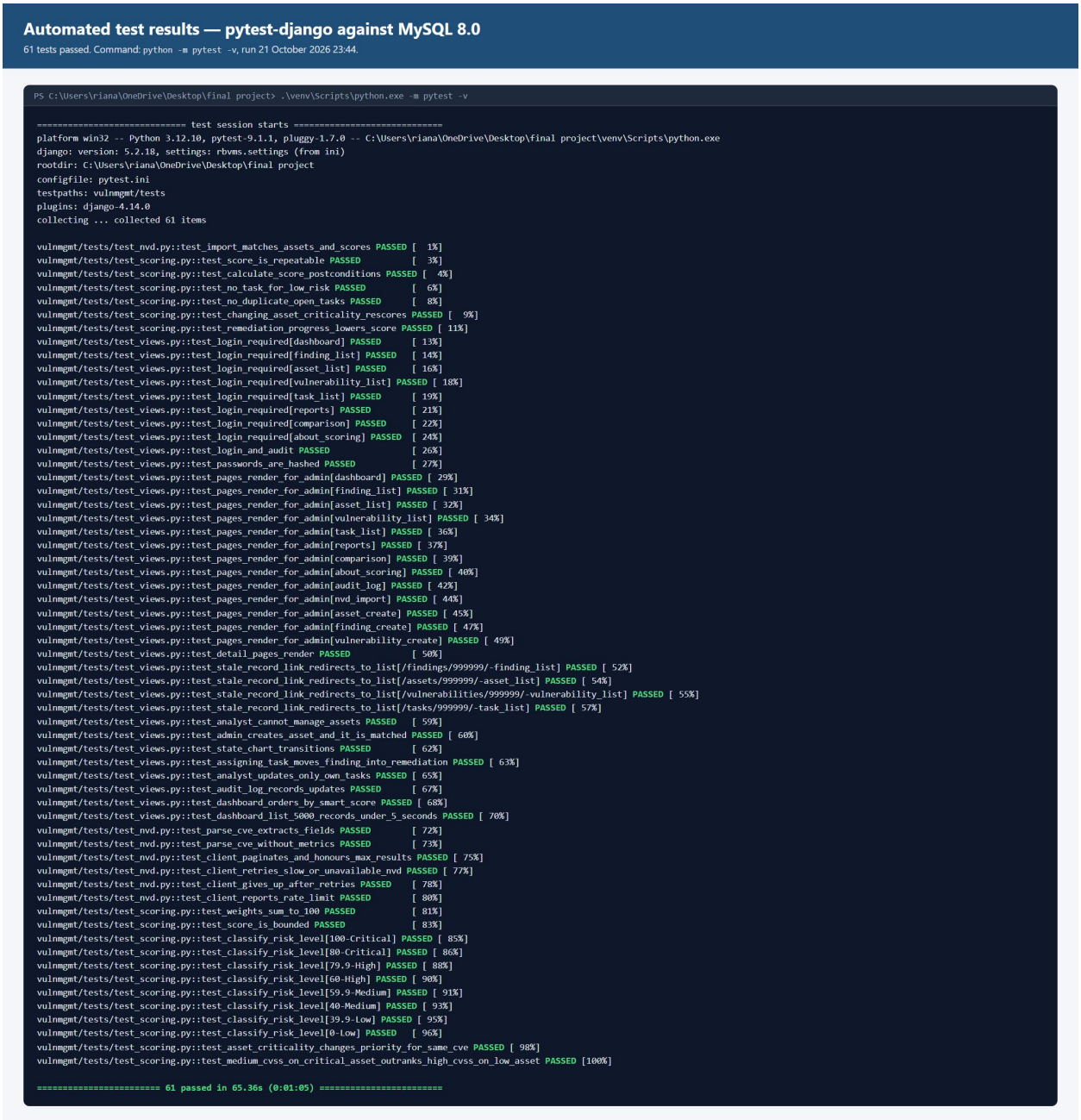


Fig 48. All 61 automated tests passing against MySQL 8.0.

Table 4. Defects found during testing.

ID	Found by	Description	Action	Status
D-01	TC-11	An optional form field left empty was logged as changed in the audit log	Comparison corrected; regression test added; retested in TC-11R	Fixed
D-02	TC-12	The 5,000-record list took 6.5 to 8.8 s to load, above the 5 s required by NFR 1	Rows built in Python, responses compressed; retested in TC-12R (3.8 to 5.8 s)	Partly fixed

4.3 Project evaluation

Table 5 evaluates the project against the objectives set in Section 1.1.2. An objective is marked as achieved only where a screenshot in Section 4.1 or 4.2 shows it working.

Table 5. Evaluation against the Chapter 1 objectives.

Objective	Status	Evidence	Comment
1. Study CVE and CVSS-based approaches	Achieved	Chapter 2; Figs 13, 14, 28 to 30	The literature review studied CVE, CVSS, EPSS and KEV, and the system uses real CVE and CVSS data from the NVD. TC-04 showed that imported scores match the NVD. EPSS was reviewed but not implemented.
2. Identify the factors that drive prioritization	Achieved	Table 1; Figs 15, 16	Asset criticality, exploit availability, exposure and remediation status are all implemented, together with CVSS, KEV status, patch availability and the number of affected systems.
3. Design a normalized relational SQL database	Achieved, with a limitation	Figs 11, 12	Six related tables with primary keys, foreign keys and unique constraints in MySQL 8.0. Two columns store lists in one field (an asset's software keywords and a CVE's affected products), which is not strictly first normal form.
4. Calculate a Smart Priority Score with weighted criteria	Achieved	Figs 15, 16, 31 to 43	All eight hand-calculated scores in TC-05 to TC-10 matched the system exactly. The weights were chosen by the developer and have not been calibrated against real incident data.
5. Implement a dashboard for prioritizing remediation	Achieved	Figs 18, 19, 35, 37	The dashboard ranks findings by Smart Priority Score, and TC-07 showed it re-ranking when an asset's risk changes.
6. Test and evaluate the system	Partially achieved	Figs 23, 25 to 48; Tables 3, 4	Twelve black-box tests, two retests and 61 automated tests were run, finding two defects. NFR 1 is not reliably met. The evaluation compared rankings for one fabricated organization only, and no user acceptance testing was done with real security administrators.

The comparison in Fig 23 supports the central claim of the project. Only five of the ten highest-priority findings are the same under the two rankings. Zerologon on the Finance DB Server rises 19 places, from 31st by CVSS alone to 12th, because it is actively exploited and sits on a critical asset, while Log4Shell on the low-value Dev/Test VM falls 17 places despite its CVSS score of 10.0. This shows that adding organizational context changes remediation priorities. However, it is a single fabricated scenario, so it demonstrates the effect rather than measuring it across organizations.

Against the requirements in Section 2.4, FR1 and FR3 to FR10 are shown working in Sections 4.1 and 4.2. FR2 is only partly met because imports run on demand but no schedule has been configured. NFR 2 and NFR 3 are met (Fig 48), and NFR 1 is not reliably met (TC-12R). Testing also showed a limitation in how CVEs are matched to assets: the match relies on an asset's software keywords appearing in the CVE's product list. The two assets that list curl were not matched to either curl CVE in the data set, because the NVD records the product as libcurl, so those findings were missed.

With more time, the following would be done next:

- Meet NFR 1 reliably by paging the prioritized list on the server or loading rows as the user scrolls, and run the system on a production web server instead of Django's development server.
- Improve asset matching by recording each asset's installed products as CPE names in a separate table, which would also remove the two multi-valued columns noted under Objective 3.
- Run imports automatically on a schedule to meet FR2 fully, and request an NVD API key to raise the rate limit.
- Add the Exploit Prediction Scoring System (EPSS) as a further factor, as reviewed in Section 2.1.
- Evaluate the score with several organizational scenarios and with real security administrators, and calibrate the weights against that feedback.
- Harden the system for deployment: turn off debug mode, serve over HTTPS, and host Bootstrap and Chart.js locally rather than from a content delivery network.

References

Cisco (2026) Cisco Security Risk Score Overview. Available at: https://www.cisco.com/c/en/us/products/collateral/security/vulnerability-management/security-risk-score-so.html (Accessed: 7 October 2026).

Cloudsmith (2025) CVSS vs EPSS: Smarter Vulnerability Risk Prioritization. Available at: https://cloudsmith.com/blog/vulnerability-scoring-systems (Accessed: 11 September 2026).

Common Vulnerabilities and Exposures (CVE) Program (n.d.) CVE Program. Available at: https://www.cve.org (Accessed: 11 September 2026).

DefectDojo (2026) DefectDojo: Open Source Vulnerability Management and ASPM. Available at: https://defectdojo.com/ (Accessed: 7 October 2026).

Elmasri, R. and Navathe, S. (2017) Fundamentals of Database Systems. Pearson.

FIRST (n.d.) Common Vulnerability Scoring System (CVSS). Available at: https://www.first.org/cvss/ (Accessed: 11 September 2026).

National Vulnerability Database (NVD) (n.d.) NVD — National Vulnerability Database. Available at: https://nvd.nist.gov (Accessed: 11 September 2026).

Pressman, R.S. and Maxim, B.R. (2020) Software Engineering: A Practitioner's Approach. McGraw-Hill.

Recorded Future (2026) Risk-Based Vulnerability Intelligence Does What CVSS Can't. Available at: https://www.recordedfuture.com/blog/risk-based-vulnerability-cvss-doesnt (Accessed: 11 September 2026).

Shimizu, N. and Hashimoto, M. (2025) 'Vulnerability Management Chaining: An Integrated Framework for Efficient Cybersecurity Risk Prioritization', arXiv preprint arXiv:2506.01220.

Souppaya, M. and Scarfone, K. (2022) Guide to Enterprise Patch Management Planning: Preventive Maintenance for Technology (NIST SP 800-40 Rev. 4). Gaithersburg, MD: National Institute of Standards and Technology.

Stallings, W. (2021) Network Security Essentials: Applications

and Standards. Pearson.